Code2LoRA: Hypernetwork-Generated Adapters for Code Language Models under Software Evolution

Liliana Hotsko · Yinxi Li · Yuntian Deng · Pengyu Nie

ABSTRACT

Code language models need repository-level context to resolve imports, APIs, and project conventions. Existing methods inject this knowledge as long inputs (retrieved through RAG or dependency analysis) or through per-repository fine-tuning and LoRA—costly at repository scale and brittle to evolving codebases. We introduce Code2LoRA, a hypernetwork framework that generates repository-specific LoRA adapters, effectively injecting repository knowledge with zero inference-time token overhead. Code2LoRA supports two usage scenarios: Code2LoRA-Static converts a single repository snapshot into an adapter, suitable for comprehension of stable codebases; while Code2LoRA-Evo maintains an adapter backed by a GRU hidden state updated per code diff, suitable for active development of evolving codebases. To evaluate Code2LoRA against parameter-efficient fine-tuning baselines, we build RepoPeftBench, a benchmark of Python repositories with two tracks: a static track with training and test assertion-completion tasks, and an evolution track with commit-derived training and commit-derived test tasks. On the static track, Code2LoRA-Static achieves % cross-repo and % in-repo exact match, matching the per-repository LoRA upper bound; on the evolution track, Code2LoRA-Evo achieves % cross-repo exact match (+ pp over a single shared LoRA).[a]

[a]Code2LoRA's code can be found at https://anonymous.4open.science/r/code2lora-6857; the model checkpoints and RepoPeftBench datasets can be found at https://huggingface.co/code2lora.

1 Introduction

Real codebases span thousands of files whose imports, APIs, and conventions a code language model must know to complete assertions, fix bugs, or navigate a project. Today's LLM-based coding assistants typically inject this repository knowledge as long inputs, in the form of retrieved relevant files through RAG (retrieval-augmented generation) or dependency analysis, and pay for the retrieved context at every query. This is costly because repository-level context can be massive, stressing the LLM's context window and RAG's retrieval capability. Another approach is

to fine-tune the model or LoRA adapters [13] for one repository or a group of related repositories, pushing knowledge into parameters. These methods also require costly training, and even worse, are brittle to *evolving* codebases, where every commit can invalidate the adapter and require retraining.

Recent work on hypernetwork-generated LoRA adapters [2, 3, 11] is promising: a single forward pass over a conditioning input produces task- or document-specific weights for a frozen LLM. These methods, however, are built for short natural-language task descriptions or single documents, not the long context a repository typically carries, and they assume a static conditioning input with no mechanism for tracking a codebase as it evolves.

We propose Code2LoRA, a hypernetwork framework that generates repository-specific LoRA adapters, effectively injecting repository knowledge with zero inference-time token overhead. We design around two orthogonal axes—*how* knowledge enters the parameters and *when* it is updated—and instantiate them as two usage scenarios: Code2LoRA-Static converts a single repository snapshot into an adapter, suitable for comprehension of stable codebases; Code2LoRA-Evo maintains an adapter backed by a GRU hidden state updated per code diff, so the recurrence augments (rather than replaces) the snapshot prior, suitable for active development of evolving codebases.

We evaluate Code2LoRA on RepoPeftBench, a benchmark of Python repositories (in-distribution and a -repository temporal holdout created after the scrape cutoff). RepoPeftBench divides each repository into non-test and test portions: the model may use non-test code as repository context and must complete assertion-completion tasks in the test portion, which is a task that requires complex reasoning capabilities [17]. Two tracks instantiate our usage scenarios: a static track with training and test tasks on a single repository snapshot, and an evolution track with training and test tasks drawn from commit history. Evaluation uses in-repo (IR) and cross-repo (CR) splits on the in-distribution corpus, plus a temporal out-of-distribution (OOD) test split on the post-cutoff holdout (§6.3).

On the static track, Code2LoRA-Static achieves % cross-repo exact match, well above context-injection methods such as RAG and dependency-resolved context; without any per-repository training it also reaches % in-repo exact match, matching the per-repository LoRA upper bound. On the evolution track, snapshot-based adaptation goes stale once evaluation uses commit-derived tasks; Code2LoRA-Evo reaches % cross-repo exact match, + pp over a shared LoRA. On the temporal OOD holdout, Code2LoRA-Evo remains the strongest method under the same commit-derived protocol (§6.3).

The main contributions of this work include:

- **Idea.** We propose using hypernetworks to effectively inject repository knowledge into code language models, and frame the problem along *how* knowledge enters model parameters and *when* it is refreshed.
- **Framework.** We design and implement Code2LoRA, a hypernetwork that maps repository code to LoRA adapters with zero inference-time token overhead, instantiated as Code2LoRA-Static (mapping one repository snapshot) and Code2LoRA-Evo (maintaining an adapter from sequential code diffs).

- **Benchmark.** We curate RepoPeftBench, a benchmark of Python repositories, including a -repository temporal holdout for out-of-distribution evaluation.
- **Evaluation.** Code2LoRA outperforms the strongest baselines on RepoPeftBench by + pp on the static track and + pp on the evolution track, with consistent gains on the temporal OOD holdout (§6.3).

2 Related Work

Parameter-efficient fine-tuning. LoRA [13] enables efficient adaptation through low-rank decomposition of weight updates; extensions include QLoRA [7], DoRA [22], weight merging [36], multi-LoRA routing [14], LoRACode [4], and MoLE [39], which trains a separate LoRA module per programming language. These methods treat adapters as static artifacts, trained per task, per language, or per repository; Code2LoRA instead *generates* adapters conditioned on repository context, enabling adaptation to unseen codebases without retraining.

Hypernetworks for LoRA generation. Hypernetworks [11] generate the parameters of a target network from a conditioning signal. Recent applications to language models include HyperTuning [27] and HyperLoRA [24] for cross-task generalization, Generative Adapter [5] for single-pass contextualization, and Zhyper [1] for factorized conditioned LoRA generation. Closest to our framework are Text2LoRA [2] and Doc2LoRA [3], which both map a whole input (a task description and a document, respectively) to a LoRA in one forward pass. Text2LoRA conditions on a short task description via an external text encoder and targets only Q/V projections; Doc2LoRA conditions on a document via per-layer activations of the frozen target LLM (Perceiver [16] encoder, MLP `down_proj` only) and is built for document QA, not code. Code2LoRA-Static generalizes this family to a third input modality—an entire code repository—and to full target coverage (all seven attention and MLP projections rather than Q/V or down-projection only). To isolate the LoRA-generation head from confounds, we strengthen Text2LoRA along both axes: we feed it the same whole-repository embedding Code2LoRA-Static consumes, and we emit LoRAs for the same seven projection types per layer. The strengthened Text2LoRA still underperforms Code2LoRA-Static, pinning down the head as the bottleneck for repository-level adaptation. Code2LoRA-Evo adds a second hypernetwork design: a GRU aggregates sequential code diffs into a hidden state that conditions adapter generation at each commit, yielding an adapter trajectory over a repository's lifetime; no analogue exists in the Text2LoRA/Doc2LoRA line of work, which only model a single static input.

Software evolution and continual code adaptation. Software evolution and mining software repository—tracking how code changes commit by commit, file by file—is a well-established line of software engineering research [12, 19], underpinning analyses of change impact, bug introduction [32], and refactoring detection [33] over long version histories. In NLP, a parallel line investigates *when* a deployed model should be refreshed: continual pretraining and online fine-tuning aim to keep language models current under temporal drift [18, 20], but typically maintain a single global checkpoint and have no notion of *which* repository is being adapted to.

Code2LoRA-Evo sits at the intersection of these two lines: it treats sequential code diffs as the unit of update and refreshes a repository-specific adapter as the commit history unfolds. This is the first hypernetwork formulation that targets repository-level adaptation under software evolution rather than a single static snapshot.

Repository-level code understanding and generation. Prior work on incorporating repository context typically routes information through the *input*: RepoFusion [31] trains with cross-file context, RepoCoder [37] iteratively retrieves and generates, RepoFormer [35] uses selective retrieval, CoCoMIC [8] jointly models in-file and cross-file context, R2C2-Coder [6] enhances repo-level completion with repository-context-aware methods, and RepoHyper [26] uses semantic-graph retrieval. Evaluation benchmarks include CrossCodeEval [9] and RepoBench [23]. Code2LoRA instead distills repository knowledge into model *parameters*, avoiding context-window limits and per-query retrieval cost, and—through Code2LoRA-Evo—tracks how that knowledge changes as code evolves commit by commit. We base our experiments on Qwen2.5-Coder-1.5B [15]; other recent code LLMs include CodeLlama [30], StarCoder [21], and DeepSeek-Coder [10].

3 Method

Code2LoRA is a hypernetwork framework that generates repository-specific LoRA adapters for a frozen code LM, effectively injecting repository knowledge with zero inference-time token overhead. As illustrated in Figure 1a, the framework has three components: a shared **repository encoder** (§3.1) that maps repository-level context to dense embeddings, a **hypernetwork** that maps those embeddings to LoRA weights, a **base LLM** that receives the generated adapter and performs inference. Only the hypernetwork is trained, via the standard language-modeling loss; the repository encoder and base LLM are frozen. The two usage scenarios differ in hypernetwork design: Code2LoRA-Static (§3.2) directly projects the repository embedding into LoRA weights; Code2LoRA-Evo (§3.3) inserts a GRU before the projection head to aggregate a sequence of diff embeddings.

3.1 Repository Encoder

Repository-level context must be compressed into a fixed-size vector before the hypernetwork can consume it. We adopt a training-free two-step embedding approach that works effectively in practice using a frozen Qwen3-Embedding-0.6B model.

Step 1: file-level embedding. Each file f_i in the repository context (or its diff Δf_i) is divided into 4096-token chunks with 512-token overlap, embedded by the frozen model, and mean-pooled over chunks to produce a file vector $\mathbf{f}_i \in \mathbb{R}^d$ (d=1024).

Step 2: repository-level aggregation. For a full repository snapshot, each file vector receives an importance weight w_i based on a combination of content distinctiveness, file size, and path importance. The repository embedding is the concatenation of a weighted mean and a max pool,

$$\mathbf{e} = \left[\textstyle\sum_i w_i \mathbf{f}_i \,;\, \max_i \mathbf{f}_i \right] \in \mathbb{R}^{2d},$$

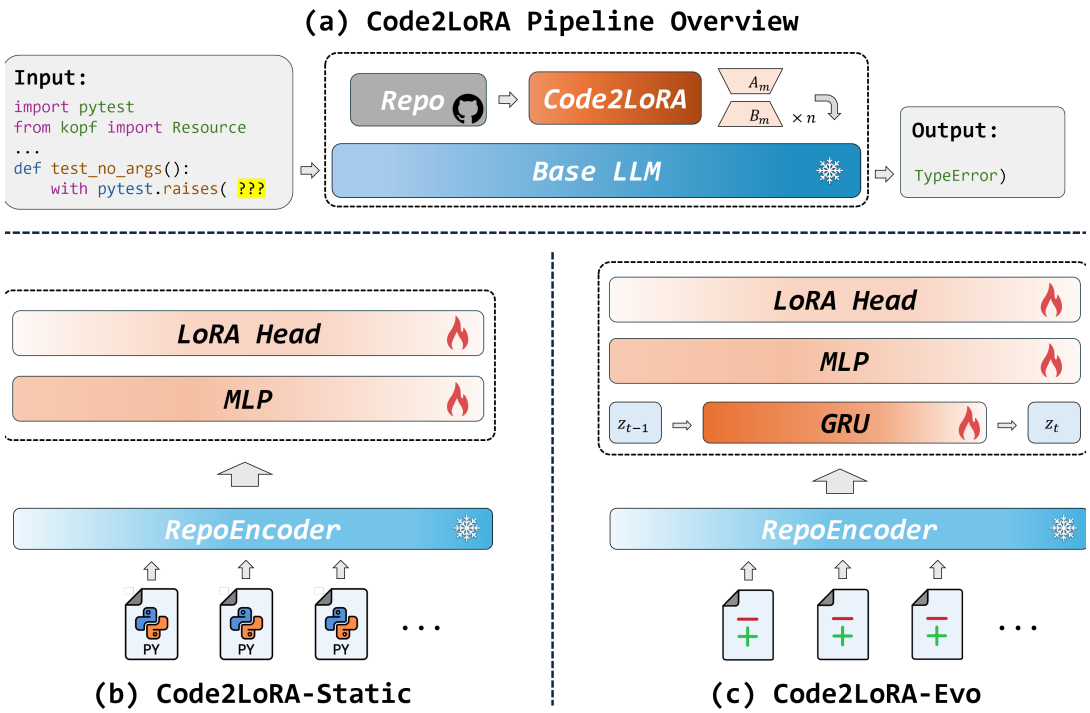

FIGURE 1 Code2LoRA architecture. **(a)** Overall pipeline: repository context is encoded and mapped to LoRA adapters, which are injected into a frozen LLM to support inference (example task: assertion completion). **(b)** Code2LoRA-Static's static hypernetwork. **(c)** Code2LoRA-Evo's recurrent hypernetwork.

capturing both the average character and the most distinctive features of the codebase. The embeddings are pre-computed at training time.

3.2 Code2LoRA-Static

The static hypernetwork in Code2LoRA-Static maps a single embedding $\mathbf{e}$ to a LoRA adapter in one forward pass. For each module type $m \in \{q, k, v, o, gate, up, down\}$, its LoRA matrices $\mathbf{A}_m$ and $\mathbf{B}_m$ are generated by a shared 2-layer MLP with GELU activation followed by dedicated output heads:

$$\begin{aligned}\mathbf{h} &= \sqrt{d_h}\,\mathrm{L2Norm}(\mathrm{MLP}(\mathbf{e})),\\ \mathbf{A}_m &= \tanh(\mathrm{Head}_m^A(\mathbf{h})) \cdot \exp(s_m^A),\\ \mathbf{B}_m &= \tanh(\mathrm{Head}_m^B(\mathbf{h})) \cdot \exp(s_m^B),\end{aligned}$$

where learnable log-scales $s_m^{A/B}$ control adapter magnitudes (initialized to -3.5). LoRA matrices are shared across all layers of base LLM and injected via $\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{B}_m\mathbf{A}_m$. With hidden dimension d_h=1024 and LoRA rank r=16, the Code2LoRA-Static hypernetwork has ∼720M trainable parameters. Code2LoRA-Static's hypernetwork architecture is similar to that of Text2LoRA [2] and Doc2LoRA [3], but (1) is driven by a whole-repository embedding summarized

from millions of tokens rather than a task description, and (2) injects LoRA to all seven module types rather than just Q/V or down-projection to be more flexible.

3.3 Code2LoRA-Evo

The recurrent hypernetwork in Code2LoRA-Evo maintains a repository-specific adapter over a chronological stream of diff embeddings $\{\mathbf{e}_t\}$. The diff embeddings are aggregated by a GRU recurrent neural network: at step t the encoder (§3.1) supplies $\mathbf{e}_t$, which is linearly projected and combined with the previous state,

$$\mathbf{z}_t = \mathrm{GRU}(\mathrm{LayerNorm}(\mathrm{Linear}(\mathbf{e}_t)), \mathbf{z}_{t-1}).$$

The initial GRU state $\mathbf{z}_0$ is initialized by a small linear projector given the initial repository embedding (e.g., on the first commit). At each step t, the LoRA adapter is generated by the shared head (§3.2) with $\mathbf{z}_t$ substituted for $\mathbf{e}$, yielding an *adapter trajectory* over the repository's lifetime. Each update requires only one GRU step on the stored diff embedding $\mathbf{e}_t$, which is substantially cheaper than re-encoding the full repository. Beyond the shared head, the GRU and initial-state projector add ∼25M parameters, for ∼745M trainable parameters in total.

3.4 Training

We train the hypernetwork end-to-end by minimizing cross-entropy on assertion-completion pairs from the frozen base LLM, with LoRA weights supplied by $\mathrm{Hypernetwork}_\theta$:

$$\mathcal{L}(\theta) = -\sum_{(x,y)\in\mathcal{D}} \log p\big(y \mid x; \mathrm{Hypernetwork}_\theta(\mathbf{u})\big),$$

where x is the input prefix, y the output target, and $\mathbf{u} = \mathbf{e}$ for Code2LoRA-Static or $\mathbf{u} = \mathbf{z}_t$ for Code2LoRA-Evo. For Code2LoRA-Evo, we optimize with truncated backpropagation through time, detaching $\mathbf{z}_t$ every K=16 steps (App. D). Batches are formed by first sampling a repository, then a pair of input-output from it, so that the hypernetwork sees diverse repositories and does not overfit to data-rich ones.

4 RepoPeftBench: A Repository-Level PEFT Benchmark

We construct RepoPeftBench, a repository-level benchmark for parameter-efficient fine-tuning of code language models. The corpus comprises Python repositories drawn from GitHub under shared quality filters—each uses `pytest` or `unittest`, carries a permissive license, and shows recent activity—partitioned along a fixed temporal cutoff (`2025-04-01`) into in-distribution repositories and a out-of-distribution (OOD) repositories. The in-distribution set was collected before the cutoff date, and requires an additional filter of having at least 300 stars (to ensure high-quality), which supplies all training and validation splits; commit histories are truncated at the cutoff date. The OOD set comprises repositories created strictly after the cutoff date and is reserved for held-out test-time evaluation only (§6.3). We collect both the last snapshot as well as the full commit histories of each repository.

Two evaluation tracks share the same task, metrics, and CR/IR repository partitions but differ in how instances are indexed in history (§4). Table 1 summarizes the split sizes used in all reported results. Benchmark construction details are in Appendix B.

Task. Each instance is an assertion-completion input-output pair: the model receives a structured prefix from a test file and must predict the expected value of an assertion. The task follows the code-execution probe of LiveCodeBench [17], but replaces hand-curated single-function snippets with assertions mined at scale from real test suites. Assertion completion is well suited to repository-level evaluation because all instances in a repository share the same non-test source as conditioning context. Repository-level code completion, as in RepoBench [23], is not suitable because each target file requires excluding that file from context to prevent leakage and thus a different repository slice per instance. CrossCodeEval [9], RepoHyper [26], and R2C2-Coder [6] likewise ship only retrieval-selected slices; RepoPeftBench releases full information of each repository to evaluate methods that ingest the full codebase.

We extract instances from five assertion families: bare `assert`, `self.assert*`, `pytest.raises`, `pytest.approx`, and NumPy-style `assert_*`. The *input* concatenates imports, the enclosing class (if any), helper methods, and the test-function body up to the assertion cut point; the *output* is the expected value of the assertion, namely the right-hand side of the binary comparison operator, or the last argument of the assertion function call.

Repository splits. We partition the in-distribution repositories into cross-repo (CR) and in-repo (IR) sets, shared by both evaluation tracks. **Cross-Repo (CR)** holds out 103 repositories entirely at training time (51 validation, test) to measure generalization to unseen codebases. **In-Repo (IR)** uses the remaining repositories for training and is the only setting in which per-repository LoRA is defined; held-out instances within each training repository are assigned by the track-specific protocol below.

Evaluation tracks. The **Static** track draws every instance from a single snapshot per repository (tasks) and corresponds to Code2LoRA-Static: on CR splits, tasks are extracted from each held-out repository's last commit snapshot; on IR splits, tasks are also extracted from last commits, and are randomly split into training, validation, and test sets in a ratio of 8:1:1. The **Evolution** track replays each repository's commit history and emits a task whenever a commit adds or modifies an assertion, storing the input-output pair together with the production-code diff Δ_t; it corresponds to Code2LoRA-Evo. On CR splits, evaluation uses all commit-derived tasks from held-out repositories; on IR splits, following the time-segmented methodology of Nie et al. [25], commits within each training repository are partitioned chronologically so that training examples strictly precede validation and test. Evolution-track training and evaluation each retain at most eight tasks per commit; Code2LoRA-Evo training further caps at four tasks per test file so that no commit dominates a backpropagation window. Table 1 reports the number of tasks for every split used in our experiments. Commit histories are bursty: repositories accumulate hundreds of test-touching commits in irregular clusters (Appendix Figure 2), which motivates streaming adaptation via Code2LoRA-Evo rather than a single frozen snapshot.

Split	Repos	Commits		
Static track				
Train	409	409	39,612	96.9
CR Val / Test	51 / 52	51 / 52	6,213 / 6,414	121.8 / 123.3
IR Val / Test	409 / 409	409 / 409	4,833 / 5,222	11.8 / 12.8
Evolution track				
Train (Code2LoRA-Static and baselines)	400†	400	44,149	110.4
Train (Code2LoRA-Evo)	400†	45,516	215,129	537.8
CR Val / Test	49 / 51	8,614 / 6,618	58,944 / 44,732	1,203 / 877
IR Val / Test	389 / 389	5,710 / 6,179	38,783 / 42,061	99.7 / 108.1
Out-of-distribution holdout				
OOD Test	92	1,950	14,813	161.0

† 9 repositories lack sufficient commit histories and are excluded from Code2LoRA-Evo training.

TABLE 1 Dataset statistics for RepoPeftBench, divided into static and evolution tracks (sharing the same set of in-distribution repositories) and out-of-distribution repositories, split into train/val/test sets under cross-repo (CR) and in-repo (IR) settings.

5 Experimental Setup

Models. The base LLM is Qwen2.5-Coder-1.5B [15], loaded in `bfloat16`; all baselines and both Code2LoRA usage scenarios share this backbone. Repository encoder uses Qwen3-Embedding-0.6B [38]. Both models are released under the Apache 2.0 license and our research use is consistent with their model cards.

Hyperparameters. Code2LoRA generate rank-r=16 LoRA adapters with α=32 for all seven attention/MLP projection types, with each $(\mathbf{A}_m, \mathbf{B}_m)$ pair shared across all 28 transformer layers (§3). Code2LoRA-Static has ∼720M trainable parameters, while Code2LoRA-Evo has ∼745M trainable parameters. We train both for 3 epochs with AdamW (cosine schedule) on a single H100 80 GB GPU using TRL [34]; full hyperparameters, schedules, and sequence-length budgets are in Appendix D.

Baselines. We evaluate against various baselines:

- : base LLM (Qwen2.5-Coder-1.5B).
- : non-test source files pre-chunked into 512-token segments, embedded with Qwen3-Embedding-0.6B; top-k retrieved chunks prepended to the prefix at inference (results for $k \in \{5, 10\}$ and chunk size 256 in Appendix C).
- : prepends function and class definitions reachable from each prefix's imports via dependency

	Cross-Repo (CR Test)			In-Repo (IR Test)		
Method	EM (%)	EditSim	CodeBLEU	EM (%)	EditSim	CodeBLEU
	45.7	0.605	0.646	46.8	0.624	0.655
	39.7	0.516	0.556	42.1	0.544	0.581
	48.2	0.625	0.657	49.5	0.640	0.667
	51.4	0.695	0.678	55.9	0.727	0.714
	53.9	0.703	0.688	56.8	0.731	0.713
	47.4	0.663	0.649	50.4	0.687	0.675
†	—	—	—	64.0	0.801	0.788
Text2LoRA	45.8	0.606	0.647	46.7	0.625	0.655
Code2LoRA-Static	**63.8**	**0.784**	**0.778**	**66.2**	**0.806**	**0.797**

†

TABLE 2 Results on RepoPeftBench static track.

analysis, with relevance-aware compression under an adaptive token budget (Appendix D.1).

- : all model parameters are made trainable.
- : one rank-16 adapter trained on *all* repositories.
- [39]: one rank-16 adapter trained *per* repository (IR splits only), serving as an upper bound on repository-level adaptation.
- Text2LoRA [2]: a hypernetwork that emits a LoRA from an external task embedding. To control for input modality and target-module coverage, we strengthen the upstream baseline along both axes: the natural-language task description is replaced with the same repository encoder that Code2LoRA uses (mean+max-pooled Qwen3-Embedding-0.6B), and the output heads are extended from $\{\mathbf{Q}, \mathbf{V}\}$ to all seven attention and MLP projections. Training data, loss, and budget match Code2LoRA, so only the LoRA-generation head differs (details in Appendix D).

Evaluation metrics. We report **Exact Match** (**EM**, after whitespace collapsing and trailing-punctuation removal, with relaxed matching that tolerates model overgeneration); **Edit Similarity** (`difflib` [28] `SequenceMatcher` ratio); and **CodeBLEU** [29], which incorporates AST and data-flow structure in addition to n-gram overlap.

	Cross-Repo (CR Test)			In-Repo (IR Test)		
Method	EM (%)	EditSim	CodeBLEU	EM (%)	EditSim	CodeBLEU
	31.5	0.490	0.515	29.3	0.469	0.501
	23.6	0.411	0.446	23.0	0.402	0.437
	31.1	0.490	0.516	31.6	0.494	0.517
	55.1	0.749	0.709	61.3	0.787	0.753
†	—	—	—	64.2	0.803	0.788
Text2LoRA	41.7	0.596	0.600	43.5	0.612	0.613
Code2LoRA-Static	55.7	0.760	0.716	60.6	0.787	0.749
Code2LoRA-Evo	**60.3**	**0.810**	**0.763**	**64.5**	**0.828**	**0.790**

†

TABLE 3 Results on RepoPeftBench evolution track.

6 Results

We organize the results around the two evaluation tracks of RepoPeftBench. The static track (§6.1, Table 2) evaluates Code2LoRA-Static and baselines on a single snapshot of each repository; Code2LoRA-Evo requires commit history and therefore does not apply to this track. The evolution track (§6.2, Table 3) evaluates all methods on commit-derived prefixes.

6.1 Static Track

Table 2 shows the results on RepoPeftBench's static track. On CR evaluation, Code2LoRA-Static reaches % EM, + pp over the strongest baseline (, %) and above every context-injection method (%, %) and other fine-tuned baselines (%, %). The strengthened Text2LoRA baseline, which matched with Code2LoRA on input modality (whole-repository embedding) and target-module coverage (all seven projections), reaches only % EM; this isolates the Text2LoRA hypernetwork as the bottleneck for repository-level adaptation, since only the LoRA-generation head differs from Code2LoRA-Static once input and targets are matched. On IR evaluation, Code2LoRA-Static reaches % EM, matching the upper bound (%) without any per-repository training—confirming that cross-repository transfer learned by the hypernetwork is more valuable than fitting one adapter per repository on the in-repo data budget.

Method	EM (%)	EditSim	CodeBLEU
	44.6	0.568	0.630
	32.6	0.464	0.536
	45.5	0.584	0.637
	72.3	0.836	0.817
Text2LoRA	60.4	0.720	0.740
Code2LoRA-Static	72.2	0.842	0.818
Code2LoRA-Evo	**74.1**	**0.866**	**0.846**

TABLE 4 Results on RepoPeftBench OOD set.

6.2 Evolution Track

Real repositories evolve commit by commit, and a static snapshot adapter goes stale once the edit stream diverges from the snapshot it was trained on. The evolution track (Table 3) evaluates with commit-derived tasks and is where Code2LoRA-Evo—with a GRU that aggregates sequential code diffs—applies.

Table 3 reports evolution-track results on commit-derived prefixes. Relative to the static track (Table 2), commit-derived tasks are substantially harder: CR EM drops from % to %. Both context-injection methods collapse: falls below the pretrained backbone on CR and IR, while recovers only to pretrained levels on CR and yields a modest IR gain. Among fine-tuned methods, reaches % / % EM; reaches 64.2% IR EM (the only applicable split). Code2LoRA-Static, included as a within-framework reference on the same commit-derived inputs, scores % / %, which is close to on CR and markedly below its static-track performance (% / %). The strengthened Text2LoRA baseline reaches only 41.7% / 43.5% EM, far below both Code2LoRA variants on this track. Code2LoRA-Evo is the strongest method on both splits (% CR, % IR EM), + pp over on CR and exceeding the upper bound on IR without per-repository training. Appendix Figure 9 (§F.3) shows that this lead persists across long commit histories, with the smallest downward drift among fine-tuned methods. Together with the static track (§6.1), these results show a consistent ordering: parametric adaptation outperforms context injection on both tracks, and recurrent aggregation over commit diffs outperforms a static snapshot once evaluation follows repository evolution.

6.3 Out-of-Distribution Generalization

The OOD set comprises repositories created strictly after the in-distribution training cutoff (`2025-04-01`) and used for held-out evaluation only, which challenges the generalization of the learned hypernetwork on new types of repository-level context. Table 4 reports results on the temporal holdout under the same commit-derived protocol as Table 3. Code2LoRA-Evo achieves the highest EM (%), ahead of Code2LoRA-Static (%) and (%). OOD assertion targets are systematically shorter than in-distribution ones (median 7 characters vs. 12–13; Appendix E), which uniformly inflates exact-match scores on this split and explains why reaches % here despite % / % on the evolution track; we therefore restrict comparison to within Table 4. On that basis, Code2LoRA-Evo leads the next-best fine-tuned adapter by ∼1.8 pp EM—narrower than the evolution-track gap (∼5 pp CR EM, Table 3) but positive and consistent across EditSim and CodeBLEU.

7 Conclusion

We introduced Code2LoRA, a hypernetwork framework that generates repository-specific LoRA adapters, effectively injecting repository knowledge with zero inference-time token overhead, and RepoPeftBench, a benchmark of Python repositories suitable for evaluating repository-level PEFT methods. The framework instantiates two usage scenarios along how knowledge enters parameters and when it is refreshed: Code2LoRA-Static, which maps a repository snapshot to an adapter for stable codebases and reaches % CR / % IR EM on the static track; and Code2LoRA-Evo, which maintains an adapter via a GRU hidden state updated on each code diff for evolving codebases and reaches % CR / % IR EM on the evolution track. Experiments on out-of-distribution repositories confirms the strong generalization capability of Code2LoRA. These results demonstrate that repository knowledge is best injected parametrically and updated to track software evolution rather than through long input context. We envision Code2LoRA as a building block will support stronger, customizable to repository-level context, and less costly AI code assistants.

Limitations

Scope of evaluation. We evaluate only on Python repositories, a single frozen backbone (Qwen2.5-Coder-1.5B), and one downstream task (assertion completion derived from naturally occurring `pytest`/`unittest` suites). The architecture is in principle language- and task-agnostic by construction (multi-language embedder, per-module-type LoRA targets shared across all layers), but extending the empirical evidence to additional languages, backbones, and downstream tasks is left to future work.

OOD target-length artifact. The % OOD EM (Table 4) may be partially inflated because assertion targets in our strictly post-cutoff OOD repositories are systematically shorter (median 7 characters) than in CR/IR test (median 12–13 characters); this confound is shared by every

OOD row and we discuss it in Appendix E. We therefore emphasize the within-OOD comparison: Code2LoRA-Evo leads the next-best fine-tuned adapter by ∼1.8 pp EM, with the direction of the effect consistent across all metrics.

Surface-level metrics. Exact match misses functional equivalence; we mitigate with EditSim, CodeBLEU, and a `pytest`-based execution probe on a runnable CR-test slice. A more semantic evaluation (e.g., executing every generated assertion against the project's test runtime) is a natural extension but was out of scope for this submission's compute budget.

Model size. The LoRA-generation hypernetwork dominates the trainable parameter count—∼720M for Code2LoRA-Static and ∼745M for Code2LoRA-Evo—and is itself a function of the backbone's projection dimensions. The evolution-track finding is therefore most directly supported at the 1.5B-parameter scale; whether recurrent aggregation over commit diffs remains necessary (or sufficient) once the backbone is much larger is an open question.

Reproducibility. Code, RepoPeftBench, and hyperparameters (Appendix D) will be released upon acceptance; all experiments run on a single H100 80 GB GPU.

Potential risks. RepoPeftBench is constructed exclusively from public permissively licensed Python repositories (Appendix B), so the dataset itself does not introduce new personal data, harmful content, or proprietary code into circulation, and we redistribute each repository under its original license terms with attribution preserved. The downstream artifact—a code language model conditioned on a repository-specific LoRA—inherits the well-understood risks of code LLMs more broadly: it can be steered to emit insecure, incorrect, or licensed-code-resembling completions, and our repository-conditioning amplifies attribution risk if a user feeds in a private repository and the generated assertions surface verbatim from training repos. We make no claims of safety for production deployment without standard mitigations (license-aware filtering of generated code, human review of generated test assertions before commit, and rejection of completions matching long verbatim training spans).

Acknowledgments

We thank Saarang Agarwal, Kyunghyun Cho, Bihui Jin, Jiale Amber Wang, Wentao Zhang, Yifan Zong and the anonymous reviewers for their comments and feedback. This work is enabled in part by support provided by Compute Ontario (computeontario.ca) and the Digital Research Alliance of Canada (alliancecan.ca). This work is partially supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) under funding reference number RGPIN-2024-04909 and RGPIN-2024-05178.

References

[1] Mohamed Hesham Ibrahim Abdalla, Zhipin Wang, Christian Frey, Steffen Eger, and Josif Grabocka. 2025. Zhyper: Factorized hypernetworks for conditioned LLM fine-tuning.

Preprint, arXiv:2510.19733.

[2] Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. Text-to-loRA: Instant transformer adaption. In *Forty-second International Conference on Machine Learning*.

[3] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-lora: Learning to instantly internalize contexts. *Preprint*, arXiv:2602.15902.

[4] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. LoRACode: LoRA adapters for code embeddings. In *ICLR 2025 Third Workshop on Deep Learning for Code*.

[5] Tong Chen, Hao Fang, Patrick Xia, Xiaodong Liu, Benjamin Van Durme, Luke Zettlemoyer, Jianfeng Gao, and Hao Cheng. 2025. Generative adapter: Contextualizing language models in parameters with a single forward pass. In *The Thirteenth International Conference on Learning Representations*.

[6] Ken Deng, Jiaheng Liu, He Zhu, Congnan Liu, Jingxin Li, Jiakai Wang, Peng Zhao, Chenchen Zhang, Yanan Wu, Xueqiao Yin, Yuanxing Zhang, Zizheng Zhan, Wenbo Su, Bangyu Xiang, Tiezheng Ge, and Bo Zheng. 2025. R2c2-coder: Enhancing and benchmarking real-world repository-level code completion abilities of code large language models. *Preprint*, arXiv:2406.01359.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Conference on Neural Information Processing Systems*.

[8] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2024. CoCoMIC: Code completion by jointly modeling in-file and cross-file context. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 3433–3445.

[9] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[10] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *Preprint*, arXiv:2401.14196.

[11] David Ha, Andrew M. Dai, and Quoc V. Le. 2017. Hypernetworks. In *International Conference on Learning Representations*.

[12] Ahmed E. Hassan. 2008. The road ahead for mining software repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57.

[13] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[14] Chengsong Huang, Qian Liu, Bill Yuchen Lin, Tianyu Pang, Chao Du, and Min Lin. 2024. Lorahub: Efficient cross-task generalization via dynamic lora composition. *Preprint*, arXiv:2307.13269.

[15] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, and 5 others. 2024. Qwen2.5-Coder technical report. *Preprint*, arXiv:2409.12186.

[16] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 4651–4664.

[17] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*.

[18] Joel Jang, Seonghyeon Ye, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, Stanley Jungkyu Choi, and Minjoon Seo. 2022. Towards continual knowledge learning of language models. In *International Conference on Learning Representations*.

[19] Huzefa Kagdi, Michael L. Collard, and Jonathan I. Maletic. 2007. A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *Journal of Software Maintenance and Evolution: Research and Practice*, 19(2):77–131.

[20] Angeliki Lazaridou, Adhi Kuncoro, Elena Gribovskaya, Devang Agrawal, Adam Liska, Tayfun Terzi, Mai Gimenez, Cyprien de Masson d'Autume, Tomáš Kociskỳ, Sebastian Ruder, Dani Yogatama, Kris Cao, Susannah Young, and Phil Blunsom. 2021. Mind the gap: Assessing temporal generalization in neural language models. In *Conference on Neural Information Processing Systems*.

[21] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, and 48 others. 2023. Starcoder: may the source be with you! *Preprint*, arXiv:2305.06161.

[22] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-decomposed low-rank

adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32100–32121.

[23] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. Repobench: Benchmarking repository-level code auto-completion systems.

[24] Chuancheng Lv, Lei Li, Shitou Zhang, Gang Chen, Fanchao Qi, Ningyu Zhang, and Hai-Tao Zheng. 2024. HyperLoRA: Efficient cross-task generalization via constrained low-rank adapters generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 16376–16393.

[25] Pengyu Nie, Jiyang Zhang, Junyi Jessy Li, Raymond J. Mooney, and Milos Gligoric. 2022. Impact of evaluation methodologies on code summarization. In *Annual Meeting of the Association for Computational Linguistics*, pages 4936–4960.

[26] Huy N. Phan, Hoang N. Phan, Tien N. Nguyen, and Nghi D. Q. Bui. 2025. Repohyper: Search-expand-refine on semantic graphs for repository-level code completion. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*, page 14–25. IEEE Press.

[27] Jason Phang, Yi Mao, Pengcheng He, and Weizhu Chen. 2023. HyperTuning: Toward adapting large language models without back-propagation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 27854–27875. PMLR.

[28] Python Software Foundation. 2024. `difflib` — helpers for computing deltas. `https://docs.python.org/3/library/difflib.html`.

[29] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *Preprint*, arXiv:2009.10297.

[30] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, and 7 others. 2024. Code llama: Open foundation models for code. *Preprint*, arXiv:2308.12950.

[31] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. Repofusion: Training code models to understand your repository. *Preprint*, arXiv:2306.10998.

[32] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *ACM SIGSOFT Software Engineering Notes*, 30(4):1–5.

[33] Nikolaos Tsantalis, Mohammad Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *International Conference on Software Engineering*, pages 483–494.

[34] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformers Reinforcement Learning.

[35] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective retrieval for repository-level code completion. *Preprint*, arXiv:2403.10059.

[36] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. TIES-merging: Resolving interference when merging models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

[37] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484, Singapore.

[38] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *Preprint*, arXiv:2506.05176.

[39] Yifan Zong, Yuntian Deng, and Pengyu Nie. 2025. Mix-of-Language-Experts Architecture for Multilingual Programming . In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, pages 200–208. IEEE Computer Society.

A Use of LLMs

We used an LLM-based writing assistant to polish grammar. All ideas, analyses, experiments, and scientific claims are our own, and we take full responsibility for the content of this work.

B Dataset Details

This section documents detailed construction process and statistics of RepoPeftBench, organized as the data flows from raw GitHub repositories to the QnA splits actually consumed by the methods in Tables 2–4: task motivation (§B.1), repository selection and licensing (§B.2), construction pipeline (§B.3), the splits used at training and evaluation (§B.4), composition by assertion family and target type (§B.5), token-length distributions (§B.6), per-repository breakdown (§B.7), and the privacy / content review (§B.8).

B.1 Motivation and Task

Why a repository-conditioned assertion task. Our assertion-completion task is directly inspired by the *code execution* task of LiveCodeBench [17], which probes whether a model can

predict the runtime value produced by a piece of code at a designated point of evaluation. Treating an assertion target as the "answer" a developer wrote down for what a piece of code *should* evaluate to at exactly that line, the prediction objective inherits the same semantics—compute, in the model's head, what this expression would resolve to in this concrete context—while replacing LiveCodeBench's hand-curated, single-function snippets with naturally occurring assertions extracted at scale from real test suites. This reframing keeps the cognitive load of the original task (multi-step, type-aware, value-level reasoning over surrounding code) and additionally couples each prediction to a full repository's API surface, naming conventions, fixtures, and domain vocabulary—turning code execution into an explicit *repository-conditioned* reasoning probe.

Why a new dataset. Existing repository-level benchmarks (RepoBench [23], CrossCodeEval [9], RepoHyper [26], R2C2-Coder [6]) ship only the slices their task consumes—a target file and a handful of retrieval-selected snippets—and discard the rest of the codebase and the Git history at release time. This is fine for input-side methods but precludes any method that must ingest the *whole* repository as parameters or as a streaming state. We therefore release each repository in RepoPeftBench *whole*: every non-test source file (for the repository representation), every test file (for assertion QnAs), and every first-parent production commit (for the evolution track's diff sequences).

B.2 Repository Selection and Licensing

In-distribution selection. The GitHub Search API was queried with `language:python license:mit stars:>=300 pushed:>=2023-01-01` together with a `pytest`/`unittest` usage filter; matching repositories were ranked by star count and downloaded in two passes (the upper pool with $\geq$ 1000 stars and a mid-range pool with 300–1000 stars), yielding the 512 in-distribution repositories used for training and CR/IR evaluation.

Temporal OOD holdout. To probe generalization beyond the training scrape, we mined an additional set of repositories with the same language, testing, activity, size, and non-fork filters but *without* the $\geq$ 300-star constraint—star-count ranges were searched from 6 upward so that enough candidates exist among repositories created strictly after `2025-04-01`. Permissive licenses (MIT and Apache-2.0) were both considered during mining; 92 repositories passed fork-chain and pytest checks and yield valid assertion pairs. Together with the in-distribution corpus, these form the 604 repositories in RepoPeftBench. Because the in-distribution query hard-filtered on `license:mit`, all 512 in-distribution repositories are MIT-licensed; the OOD holdout may include Apache-2.0 repositories where that was the upstream license. We retain a copy of each repository's `LICENSE` file alongside the source tree in the released dataset, and the dataset release itself is distributed under the same MIT terms with attribution to the upstream maintainers preserved.

Intended use and consistency with upstream terms. Using the source contents of MIT-licensed public repositories for research on code language models is consistent with the upstream license, which explicitly permits use, modification, and redistribution provided that the copy-

right notice is included. RepoPeftBench and the released Code2LoRA checkpoints are intended exclusively for non-commercial research on repository-level adaptation of code LMs; downstream commercial or product deployment is out of scope for this release and would require an independent re-licensing review of each contributing repository. Derivatives produced from the dataset (e.g., embeddings, generated LoRAs, predictions) inherit the same research-use scope.

B.3 Construction Pipeline

Test file identification. Files are classified as test files if they match any of: `test_*.py`, `*_test.py`, or reside in directories named `tests/`, `test/`. Identified test files are moved to a separate `TEST_HYPERNET/` directory within each repository, preserving relative paths.

Structured prefix construction. Each QnA prefix is constructed as follows:

1. All import statements from the test file.
2. The enclosing class definition (if the test is a method).
3. Helper methods (`setUp`, `tearDown`, fixtures).
4. The test function signature and body up to the assertion cut point.

This structured approach preserves the most informative context while managing token budget.

Quality filters applied.

- Targets starting with comma (malformed AST segmentation).
- Targets outside function bodies (module-level assertions).
- Empty or whitespace-only targets.
- Duplicate targets within the same test function.
- Targets containing only punctuation or single characters.

Bursty commit pattern. Figure 2 shows the per-repository test-touching commit distribution that motivates the evolution track: test-touching commits arrive in irregular bursts rather than at uniform intervals, so a single static snapshot of any repository fails to capture the full history of assertion edits seen during active development.

B.4 Splits Used in Experiments

Table 1 (in the main paper) reports the exact splits consumed by every number in Tables 2–4 (one row per split actually used at training or evaluation time); Table 5 below expands that overview with per-commit and per-repository densities, including the smart-cap output for Code2LoRA-Evo training. For the evolution track we enforce a per-commit cap of $\leq$8 QnAs at both training time (as part of the smart cap, which additionally bounds at $\leq$4 QnAs per test file) and evaluation time: every evaluator scores the first $\leq$8 QnAs per (repo, commit) group so that the EM / EditSim / CodeBLEU averages are not dominated by a few unusually large commits with hundreds of QnAs in a single test file. The average density after the eval-time cap is $\sim$6.8 QnAs per commit (below the cap because many commits naturally have fewer than 8 QnAs).

FIGURE 2 Bursty commit pattern, illustrated using randomly selected 5 repositories out of the RepoPeftBench repositories. Test-touching commits arrive irregularly; the median repository accumulates over 100 such commits, motivating per-commit (rather than one-shot) adaptation under software evolution.

B.5 Composition by Assertion Family and Target Type

To characterize the assertion-completion task at the level of what the model actually predicts, Table 6 breaks down each split by assertion family (which keyword triggers the test) and by target type (what the assertion expects). The three splits are tightly aligned: bare `assert` accounts for ∼82–86% of pairs and the target distribution (numeric/string literals, variables, function calls, complex expressions) varies by at most ∼2 pp between train, CR test, and IR test. This rules out distribution shift across splits as an explanation for the cross-repo gap, and confirms that improvements on CR test are genuine generalization rather than reweighting of easier target categories.

B.6 Token-Length Statistics

Table 7 reports token-length distributions for the four input components (repository, DRC context, structured prefix, target) over the 62,294 static-track QnAs (Qwen2.5-Coder-1.5B tokenizer; same denominator as Table 6). Repositories are large (median 165K tokens), DRC context—when present—is moderate (median 517 tokens) but heavy-tailed, prefixes are compact (median 224 tokens), and targets are short (median 3 tokens). Figure 3 plots the prefix-only and DRC+prefix length distributions side by side and marks common context-window sizes, illustrating why DRC training requires the 8K-context setting of Table 9.

Split	Repos	Commits	QnAs	Cmts / repo	QnAs / cmt	QnAs / repo
Static track — one anchor snapshot per repository (no per-commit cap)						
Train	409	409	39,612	1.00	96.9	96.9
CR Val	51	51	6,213	1.00	121.8	121.8
CR Test	52	52	6,414	1.00	123.3	123.3
IR Val	409	409	4,833	1.00	11.8	11.8
IR Test	409	409	5,222	1.00	12.8	12.8
OOD Test	92	92	9,942	1.00	108.1	108.1
Evolution track — multi-commit; ≤8 QnAs / commit at train (smart-cap, ≤4/file) and eval						
Train (Code2LoRA-Static, anchor)	400	400	44,149	1.00	110.4	110.4
Train (Code2LoRA-Evo, 8-cap)	400	45,516	215,129	113.79	4.73	537.8
CR Val	49	8,614	58,944	175.80	6.84	1,203
CR Test	51	6,618	44,732	129.76	6.76	877
IR Val	389	5,710	38,783	14.68	6.79	99.7
IR Test	389	6,179	42,061	15.88	6.81	108.1
OOD Test	92	1,950	14,813	21.20	7.60	161.0

TABLE 5 Fine-grained statistics for every split actually consumed by the main tables. *Static track*: one anchor snapshot per repository (rows feed Table 2). *Evolution track*: multi-commit prefixes (rows feed Tables 3 and 4); the smart cap (≤4 QnAs per test file, ≤8 per commit) is applied to Code2LoRA-Evo training rows so that no commit can dominate a backprop window.

B.7 Per-Repository Performance Breakdown

To support repository-by-repository scrutiny of every method, we release a per-repository table covering all 409 IR-test repositories with EM, EditSim, CodeBLEU, and example counts for pretrained, FFT, sLoRA, per-repo LoRA, and Code2LoRA-Static. The supplementary materials contain the full table; aggregate distributions and the data-sparsity scatter for per-repo LoRA are summarized in Figures 6 and 7.

B.8 Privacy and Content Review

The dataset contains only non-test source files and test files from public open-source projects with permissive licenses, copied verbatim from the upstream repositories. No private repositories, user accounts, commit messages, issue bodies, or PR discussions are included; identifying information is therefore limited to whatever the upstream maintainers chose to embed in public Python source (e.g., author docstrings, copyright headers in `LICENSE` files, contact emails inside module-level docstrings of well-known libraries). We did not perform automated PII scrubbing because (i) the dataset is a redistribution of already-public, license-permitted source, and (ii) any aggressive scrubbing would alter the very identifiers (function names, fixture names, class names) that the benchmark task requires the model to predict. We did not observe offensive content in random

Property	Train	CR Test	IR Test
Assertion types			
`assert`	82.5%	86.2%	82.2%
`self.assert*`	13.5%	10.0%	13.6%
`pytest.*`	4.1%	3.8%	4.3%
Target types			
Numeric literal	18.7%	19.9%	19.4%
String literal	18.2%	18.2%	18.5%
Variable	21.7%	21.9%	21.8%
Collection	11.8%	10.2%	11.3%
Function call	9.4%	10.2%	8.9%
Complex expression	14.5%	14.0%	15.0%
Bool/None literal	5.8%	5.5%	5.1%

TABLE 6 Composition of the static-track QnAs by assertion family (which keyword triggers the test) and target type (what the assertion expects), computed over the 62,294 QnAs actually used at training and evaluation time (sum of static train, CR Val/Test, and IR Val/Test rows in Table 1). Splits are tightly aligned: every target-type fraction differs by at most ∼2 pp between train, CR test, and IR test.

spot checks of the dataset, which is consistent with the high-star permissive-license selection criterion; users who identify problematic content in any of the released repositories may file an issue against the dataset repository for removal.

C Additional Ablation Studies

C.1 RAG with Different k

We sweep the number of retrieved chunks k and chunk size to confirm that the RAG result in the main table (k=3, 512-token chunks) is the strongest configuration for our setting, and that the degradation under RAG is not an artifact of a particular budget. Pretrained RAG monotonically degrades with k on both CR and IR (Table 8, top): going from k=3 to k=10 at 512-token chunks drops CR EM by 3.4 pp and IR EM by 2.7 pp. Smaller (256-token) chunks at the same retrieval budget are uniformly worse than the 512-token variant. Combining RAG with trained adapters (Table 8, bottom) helps FFT mildly but hurts sLoRA, consistent with the finding that retrieval-injected tokens shift the distribution away from what the adapter was trained on. The largest single k used at training and reported in Table 2 is therefore the optimal RAG configuration, not a strawman.

Component	Mean	Med.	Std	p75	p95	p99	Max
Repo size	284,509	165,376	363,914	311,729	1,028,703	1,865,509	2,994,853
DRC context†	1,900	517	6,243	1,634	7,849	20,826	574,001
Prefix	360	224	566	396	992	2,588	27,171
Target	4.8	3.0	10.2	5.0	14	43	290

† Computed over 39,902 pairs (64.1%) with resolvable dependency context.

TABLE 7 Token length statistics across the 62,294 static-track QnAs (Qwen2.5-Coder-1.5B tokenizer). Repo size is the total token count of all Python source files per repository (repeated per pair). DRC statistics are over the 64.1% of pairs with resolvable dependency context.

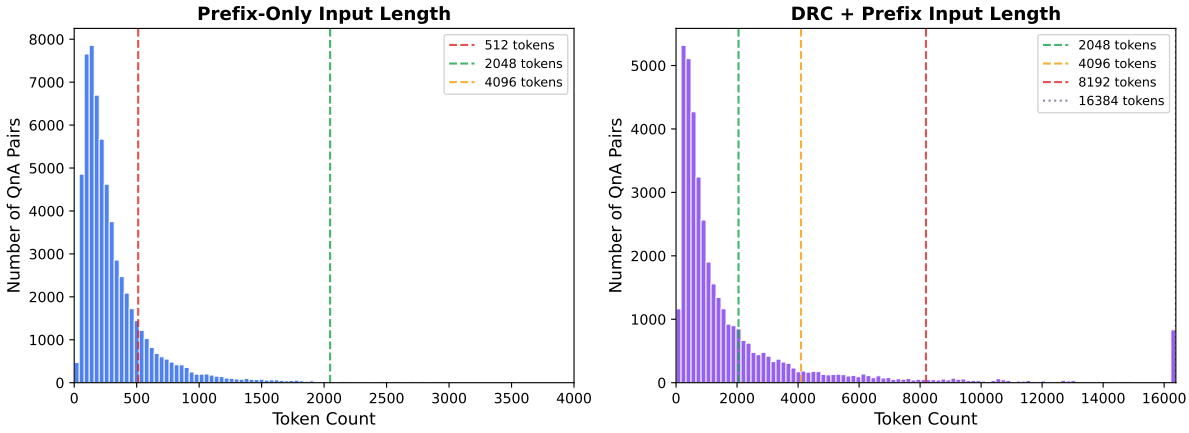

FIGURE 3 Token length distributions for prefix-only (left) and DRC+prefix (right) input formats across all splits. Vertical dashed lines mark common context window sizes. Prefix-only inputs are compact (median 224 tokens), while DRC+prefix inputs have a heavy right tail requiring larger context windows.

D Implementation Details

This section documents the dependency-resolved context (DRC) extraction algorithm and the exact hyperparameters used to train every method in Tables 2–4. All training and evaluation runs use a single H100 80 GB GPU; total compute is summarized at the end of the section.

D.1 Dependency-Resolved Context Construction

DRC takes a test prefix and, via static import analysis, returns the function and class definitions reachable from its imports. We describe the resolution strategy, the relevance-aware compression that fits results into the adaptive 8K-token budget, and the empirical coverage on RepoPeftBench.

Import resolution strategy. For each import in the test prefix:

1. Parse using AST with fallback regex for syntax errors.

		CR Test			IR Test		
Chunk	k	EM	EditSim	CB	EM	EditSim	CB
Pretrained + RAG							
512	3	39.7	0.516	0.556	42.1	0.544	0.581
512	5	37.5	0.486	0.527	41.0	0.524	0.559
512	10	36.3	0.469	0.509	39.4	0.521	0.574
256	5	35.0	0.457	0.499	38.0	0.489	0.528
256	10	33.0	0.428	0.470	35.5	0.453	0.494
Trained + RAG							
256	5 (FFT)	53.9	0.703	0.688	56.8	0.731	0.713
256	5 (sLoRA)	37.0	0.588	0.586	39.0	0.620	0.609

TABLE 8 RAG ablation over chunk size and k on CR and IR test. Top: pretrained + RAG; bottom: trained models + RAG at inference.

2. Resolve the module to a file path, trying multiple source roots: repository root, `src/`, `lib/`, package directories with `__init__.py`.
3. For relative imports, resolve relative to the test file's location.
4. If the imported name is used in the test prefix, extract its definition (function or class) from the source file via AST.

Coverage.DRC context is available for 70.3% of CR-test pairs, 64.7% of IR-test pairs, and approximately 64% of training pairs. When present, DRC adds a median of 517 tokens (mean 1,900, p95 7,850 tokens). Pairs with no resolvable imports (e.g., testing third-party libraries or built-in functions only) receive no DRC augmentation and are trained and evaluated on the plain prefix.

D.2 Detailed Architecture Diagrams

Figure 4 and Figure 5 expand the overview in Figure 1 with step-by-step training and inference details for each usage scenario.

D.3 Training Details

Table 9 lists the optimizer, schedule, sequence length, batch size, and adapter configuration for every trained baseline (FFT, sLoRA, per-repo LoRA) and for Code2LoRA-Static (with and without training-time DRC) on the static track. All methods share the same backbone (Qwen2.5-Coder-1.5B, bf16), the same optimizer (AdamW, cosine schedule, weight decay 0.01), and roughly the same effective compute budget; the methods differ in LR, sequence length, and (for adapter methods) LoRA rank, dropout, and module coverage. Code2LoRA-Static uses an 8K

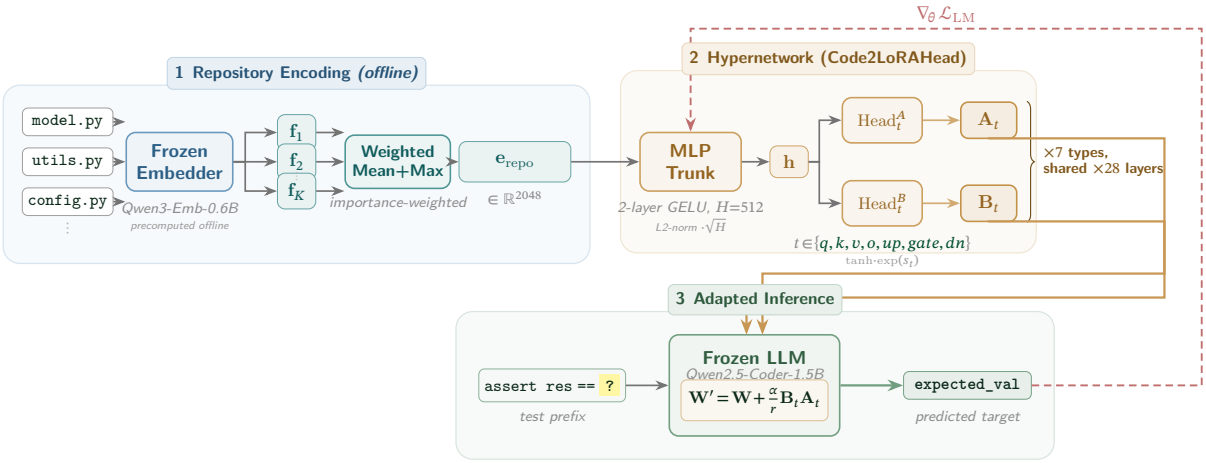

FIGURE 4 Detailed Code2LoRA-Static architecture. **(1)** Repository-level context is encoded by a frozen embedding model (Qwen3-Embedding-0.6B) and aggregated into a 2048-dim repository embedding $\mathbf{e}_{\text{repo}}$; the result is stored in the dataset and consumed verbatim at training time—gradients never flow back through the embedder. **(2)** A shared MLP trunk (2-layer GELU, hidden H=512) maps $\mathbf{e}_{\text{repo}}$ to a hidden representation $\mathbf{h}$ (L2-normalized, rescaled by $\sqrt{H}$); separate Head_m^A, Head_m^B heads emit $\mathbf{A}_m, \mathbf{B}_m$ for each of the 7 projection types via $\tanh \cdot \exp(s_m)$ scaling with a clamped learnable log-scale s_m. The same $(\mathbf{A}_m, \mathbf{B}_m)$ pair is shared across all 28 transformer layers. **(3)** Generated LoRA weights are injected into the frozen LLM via $\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{B}_m\mathbf{A}_m$. Only the hypernetwork parameters θ are trained via the language-modeling loss (dashed red); the LLM and embedder stay frozen.

sequence length to accommodate dependency-resolved context when enabled; Code2LoRA-Evo truncates BPTT every 16 commits and uses a 4K sequence length per step (§D.5).

D.4 Compute Resources

All experiments were conducted on a single NVIDIA H100 80 GB GPU per job. Total GPU hours: FFT variants ∼6 h, sLoRA variants ∼10 h, Code2LoRA-Static (no DRC) ∼17 h, Code2LoRA-Static+DRC ∼18 h, per-repo LoRA (∼0.1 h per repo × 409 repos) ∼41 h, and evaluation jobs ∼30 h. Code2LoRA-Evo training requires an additional ∼24 h on the commit-derived dataset.

D.5 Hypernetwork Training Hyperparameters

The Code2LoRA-Static variant uses input dim 2,048 (mean+max repository embedding), trunk hidden H=512, LoRA rank r=16, α=32, and all seven attention/MLP projection types shared across all 28 transformer layers. Code2LoRA-Evo uses a 1-layer GRU with hidden size 2,048 and a small *repo-state initializer* (Linear → GELU → LayerNorm) that maps the initial 2048-dim repository embedding to $\mathbf{h}_0$; the LayerNorm-ed final state $\mathbf{h}_T$ feeds into Code2LoRA-Evo's projection head (analogous in design to Code2LoRA-Static's, with trunk hidden 1,024 vs. 512). Truncated BPTT detaches the hidden state every K=16 commits. Both variants are trained for 3 epochs with AdamW (cosine schedule, weight decay 0.01): Code2LoRA-Static at LR 1×10^{-4}

	FFT	sLoRA	pLoRA	Code2LoRA-Static	+DRC
LR	2e-5	5e-5	2e-4	1e-4	(same)
Epochs	3	5	3	3	(same)
Max seq len	2,048	2,048	2,048	8,192	(same)
Batch size	4	4	4	1	(same)
Grad accum	8	8	4	8	(same)
Effective batch	32	32	16	8	(same)
LoRA rank	—	16	16	16	(same)
LoRA alpha	—	32	32	32	(same)
LoRA dropout	—	0.1	0.0	—	—
Warmup ratio	0.05	0.10	0.10	0.03	(same)
Max DRC tokens	—	—	—	—	4,096
Precision	bf16	bf16	bf16	bf16	bf16
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW
LR schedule	cosine	cosine	cosine	cosine	cosine

TABLE 9 Training hyperparameters. The "+DRC" column shares all settings with Code2LoRA-Static and adds a 4K-token dependency-resolved context budget injected ahead of the prefix. The commit-derived results in Tables 3–4 use analogous V2 trainers (1 epoch, batch 1, grad-accum 16, max seq 4,096); see §D.5 and the released code for full details.

and max sequence length 8,192; Code2LoRA-Evo at LR 5×10^{-5} and max sequence length 4,096. Best checkpoint is selected by CR-val loss.

E OOD Evaluation Caveats

Two confounds in Table 4 are worth surfacing. *(i) Prefix shape.* Table 4 uses commit-derived prefixes (median ∼7.9 KB), identical to Table 3 and *not* the short static prefixes (∼0.9 KB) of Table 2; OOD-vs-Table 3 deltas are therefore unconfounded by prefix shape, and only the underlying repositories differ. *(ii) Target length.* OOD assertion targets are systematically shorter (median 7 chars) than CR/IR-test (12–13 chars), inflating exact-match credit on every OOD row uniformly; sLoRA's OOD EM (72.3%) substantially exceeds its in-distribution EM (55.1/61.3%) for this reason. The within-table Code2LoRA-Evo vs. sLoRA gap on OOD is +1.8 pp—narrower than the in-distribution gap (+5.2/ + 3.2 pp, Table 3) but always positive, so Code2LoRA-Evo remains the best method on every split under matched inputs. We interpret the narrower OOD margin as evidence that part of the streaming advantage is recovered from within-distribution edit patterns seen at training: the OOD repositories were created strictly after the scrape cutoff, so their early-life commit trajectories were never observed.

F Broader Analysis

This section complements the main-paper analysis with the supporting figures and tables: per-repository variance and data-sparsity scatter (§F.1), the repository-count scaling curve (§F.2), the per-commit-position trend (§F.3), structural analysis of the generated LoRAs (§F.4), the LiveCodeBench-style error taxonomy and qualitative examples (§F.5, §F.6), DRC coverage broken out by availability (§F.7), and the efficiency comparison (§F.8).

F.1 Per-Repository Performance and Data Sparsity

The aggregate IR-test EM in Table 3 hides substantial repository-to-repository variance for per-repo LoRA. Across the 389 repositories evaluated by every method, per-repo LoRA EM spans the full $[0, 100]$% range with a median of 62.5% and a standard deviation of 20.9; on 10.5% of repositories (41/389) per-repo LoRA scores *below* the pretrained baseline (per-repo median 30.7%). The dominant driver is training-data availability: per-repo LoRA overfits to small in-repo datasets and frequently regresses below the unadapted backbone whenever the in-repo training pool is thin. Code2LoRA-Static sidesteps this failure mode through cross-repository knowledge transfer: the hypernetwork learns shared patterns from 409 repositories (39,612 examples) and regularizes the generated adapters, yielding the tighter per-repository EM distribution shown in Figure 6 (σ=16.8 for Code2LoRA-Static and 15.8 for Code2LoRA-Evo vs. 20.9 for per-repo LoRA; only 1.3% and 1.8% of repositories fall below pretrained, respectively) and the flatter EM-vs-data-size profile shown in Figure 7.

F.2 Repository-Count Scaling

To understand whether the hypernetwork benefits from *breadth* (more distinct repositories) or merely *depth* (more pairs), we sweep the number of training repositories at $\{10, 25, 50, 100, 150, 200, 409, 500, 623\}$ while keeping the per-repo data budget and training schedule fixed. Two findings emerge. First, with only 10 repositories (∼2% of the full training set), Code2LoRA-Static already reaches 57.7% CR-test EM—above FFT trained on the full data (51.4%, Table 2). Second, CR-test EM scales log-linearly with repository count up to ∼200 repositories and is essentially flat between 200 and 623, suggesting that breadth saturates around a few hundred distinct codebases at the current backbone size. Figure 8 plots the curve; Table 10 reports the underlying numbers.

F.3 Per-Commit Position Trend

To verify that Code2LoRA-Evo's evolution-track advantage is not driven by a few late-history commits, we plot CR-test EM as a function of each commit's normalized position within its repository's chronological history. For every repository the timeline is rescaled to $[0\%, 100\%]$ (so 0% is the first scored commit and 100% the last), QnAs are bucketed into 5%-wide bins, and each bin's score is the QnA-weighted mean EM across that bin. This collapses short and long repository histories onto a single axis and visualizes the entire lifecycle of every repository rather than only its first commits. Figure 9 shows that Code2LoRA-Evo's lead persists across

Training Repos	% of Full	CR Test EM (%)
10	2%	57.7
25	4%	60.9
50	8%	60.9
100	16%	61.3
150	24%	61.5
200	32%	62.2
409	66%	63.8
500	80%	61.2
623	100%	63.5

TABLE 10 Effect of training-repository count on CR-test EM.

the entire history; the snapshot-based methods (Code2LoRA-Static, sLoRA, FFT) exhibit the steepest downward drift, consistent with the staleness mechanism described in §6.2, while Code2LoRA-Evo stays flattest.

F.4 Structure of the Generated LoRAs

A natural question is whether the hypernetwork emits genuinely repository-specific adapters or whether it converges to a single mean adapter that happens to behave well on average. We probe this from two angles. *Diversity of adapters*: pairwise cosine similarities between the 52 mean-centered CR-test LoRAs (659K-dim flattened) span the full $[-1, +1]$ range with mean 0.01 and standard deviation 0.94, so the adapters are not a collapsed mean. *Semantic structure*: a t-SNE projection of those adapters (Figure 10) shows that repositories with similar codebases cluster together and that clusters carry coherent EM ranges, indicating that the hypernetwork's adapter manifold is smooth and semantically organized rather than arbitrary. *Per-module concentration*: a comparison of per-module weight norms (Figure 11) reveals that Code2LoRA-Static concentrates updates on a repository-specific subset of modules (typically `gate` and `up` projections), whereas FFT+DRC applies a uniform delta across all modules—a qualitative difference that helps explain Code2LoRA-Static's stronger cross-repo transfer.

F.5 Error Analysis

We classify all 2,321 incorrect CR-test predictions of Code2LoRA-Static using a LiveCodeBench-inspired taxonomy [17]. The breakdown in Figure 12 shows that no single failure mode dominates: *wrong literal* (31.0%) and *syntax error* (28.0%) together account for ∼60% of errors, with the remainder split among *type mismatch* (19.0%), *near-miss* (10.8%), and *wrong identifier* (10.2%); hallucinations and empty outputs are each under 1%. The wrong-literal class is dominated by numeric tests where the correct value depends on runtime state (e.g., expression-valued

assertions); the near-miss class corresponds to syntactically valid completions that differ from the reference only in trailing punctuation or single tokens.

F.6 Qualitative Examples

We complement the aggregate numbers with qualitative views of CR-test predictions. Figure 13 pairs two representative successes from *inline-snapshot* and *ALNS* where Code2LoRA-Static recovers repo-specific identifiers and conventions that pretrained Qwen2.5-Coder and full fine-tuning miss. We then zoom in on a representative case with an expanded layout that shows the metadata header, full test prefix, retrieved repository context, and side-by-side per-method predictions: Figure 14 illustrates the *context-quality bottleneck* case, where retrieval surfaces the relevant class definition but only the parametric methods complete the value-level reasoning step.

Beyond the two short panels of Figure 13, we feature one additional commit-derived CR-test case in full detail below. The case is drawn from the supplementary file `positive_analysis.md` (10 cases total, 5 per category) and demonstrates the complementary *context-quality bottleneck* phenomenon: RAG@3 / DRC retrieval surfaces the exact class definition that determines the assertion's outcome, yet pretrained, RAG, DRC, and sLoRA all fail to translate that prepended evidence into the correct prediction; only the hypernetwork variants complete the value-level reasoning step from the retrieved evidence.

We further feature four detailed qualitative examples drawn from the commit-derived IR-test set (GRU dataset variant; the source HTML report `report_gru_ir_test_qnas.html` samples 300 QnAs across 18 methods). Each figure shows the full test prefix, the actual DRC and RAG@3 contexts that were injected at evaluation time (trimmed to the most relevant signatures and class initializers; non-essential method bodies are elided with "`...`"), and the per-method predictions for the five methods that the report tracks for Table 3: Code2LoRA-Static, Code2LoRA-Evo, RAG, DRC, and Text2LoRA. Figures 15 and 16 are easy cases where the local prefix already exposes the completion pattern and retrieval merely corroborates it. Figure 17 is a *retrieval-precision* case: only DRC retrieves the discriminating `-> bool` signature, RAG misses it and collapses onto the n-gram-likely `is 1`; the parametric Code2LoRA variants succeed without context. Figure 18 is a *retrieval-degeneracy* case: DRC retrieves the literal answer `JobOutcome.abandoned` in a docstring and RAG retrieves the `JobOutcome` enum class plus the dotted access pattern (one inference hop away from the answer), yet both methods collapse onto a FIM-token artifact at the very first generated token; only the methods that bake the repository signal into the parameters complete the assertion.

F.7 Effect of Dependency-Resolved Context Coverage

DRC is only meaningful when the imports in the test prefix actually resolve to repository code. On CR-test, **70.3%** of pairs (4,511/6,414) have non-empty DRC, while the remaining **29.7%** (1,903 pairs) import only from the standard library or third-party packages and therefore receive no DRC augmentation. To check whether DRC's modest aggregate gain reflects a strong effect on the resolvable subset or a uniformly weak effect, we partition CR-test by DRC

	CR Test EM (%)	
Method	w/ DRC (70.3%)	w/o DRC (29.7%)
	48.1	51.5
	49.9	44.2
Code2LoRA-Static	**67.0**	**66.9**

TABLE 11 CR-test EM partitioned by DRC availability. DRC helps only when context is resolvable (+1.8 pp vs. pretrained); Code2LoRA-Static performs consistently regardless, showing the repository embedding captures information beyond import resolution.

availability in Table 11. DRC adds +1.8 pp over pretrained *only* on the resolvable subset and is actively destructive (−7.3 pp) on the no-DRC subset, where the model is forced to attend to empty context slots. Code2LoRA-Static is essentially flat across the two partitions (67.0 vs. 66.9 EM), showing that the learned repository embedding captures information beyond what import-resolved definitions provide.

F.8 Deployment Efficiency

Table 12 compares the deployment cost of every method along three axes that matter when scaling to many, continuously-changing repositories: extra inference tokens, per-repository adaptation time, and incremental storage on top of the shared frozen base model. RAG and DRC both incur per-query token overhead in the 500–2,000 range, while FFT requires ∼4 h of training and a full 3.1 GB model copy per repository. Code2LoRA-Static and Code2LoRA-Evo sit at the other extreme: zero extra inference tokens, sub-10 ms adapter generation, and bounded extra storage (679 MB for the Code2LoRA-Static hypernetwork shared across all repositories, 65 MB for the Code2LoRA-Evo variant, both *independent* of repository count). Per-repo LoRA matches the inference cost of Code2LoRA but requires ∼5 min of training per new repository and 32 MB per repository, neither of which scales.

G Discussions

We organize the discussion around three central questions raised by the framework.

Q1. Why parameters over context?For assertion completion the answer depends on a short window of repository-specific symbols rather than long-range token-level reasoning. RAG and DRC inject related but locally noisy tokens that shift the model's distribution; FFT collapses repository signal into one "average" specialization. Code2LoRA routes the same information into per-repository LoRA *parameters*, conditioning the model at every layer without paying tokens or sharing capacity across repositories—explaining the consistent gaps to FFT, DRC, and pLoRA on both IR and CR (Table 2).

Method	Extra Tokens	Adapt. Time	Extra Storage
	0	N/A	—
	∼1,500	per query	+chunk index
	∼500–2,000	per query	+import cache
	0	∼4h	+3.1 GB
	0	∼2h	+32 MB
	0	∼5 min/repo	+32 MB/repo
Code2LoRA-Static	**0**	<10ms/repo	+679 MB
Code2LoRA-Evo	**0**	<10ms + GRU enc.	+65 MB

TABLE 12 Efficiency comparison. Extra storage is beyond the shared frozen base model (Qwen2.5-Coder-1.5B, 3.1 GB in bf16). Both Code2LoRA variants add zero inference tokens and generate repo-specific adapters in a single forward pass.

Q2. Why two usage scenarios rather than one?The *how*/*when* framing admits two ends: one-shot snapshot adaptation vs. incremental refresh under evolution. Code2LoRA-Static is sufficient—and, in raw CR/IR EM, optimal—on the static track (Table 2): the same code embedding goes into a single forward pass and out comes one LoRA per module type, with no recurrence and no commit history to maintain at deployment. Real codebases, however, do not stand still: the bursty commit pattern in Figure 2 shows that snapshot adaptation accumulates staleness as a repository accumulates edits, and Table 3 shows the same Code2LoRA-Static model dropping back to parity with the single-adapter baseline once the evaluation prefix reflects commit-time state. Code2LoRA-Evo is the shared-head extension for this drift: the static head is reused, but the head's context vector becomes a recurrent hidden state updated at each recurrent step with amortized constant work per update. The two usage scenarios therefore correspond to stable-codebase comprehension vs. active development on evolving codebases, not competing ablations.

Q3. Where does Code2LoRA-Evo's edge come from?Code2LoRA-Evo reuses Code2LoRA-Static's LoRA-generation head; the only added capacity is a GRU recurrence over sequential diff embeddings before the shared MLP trunk. The empirical lead (Table 3, + pp commit-CR EM over single LoRA) is the value of aggregating edit history into the hypernetwork context commit-by-commit, rather than asking a single snapshot embedding to capture both code and its history. Results on the temporal OOD holdout in RepoPeftBench corroborate generalization (§6.3). Appendix Figure 9 corroborates this: Code2LoRA-Evo's advantage persists across the entire commit timeline, with the shallowest staleness drift among trained adapters.

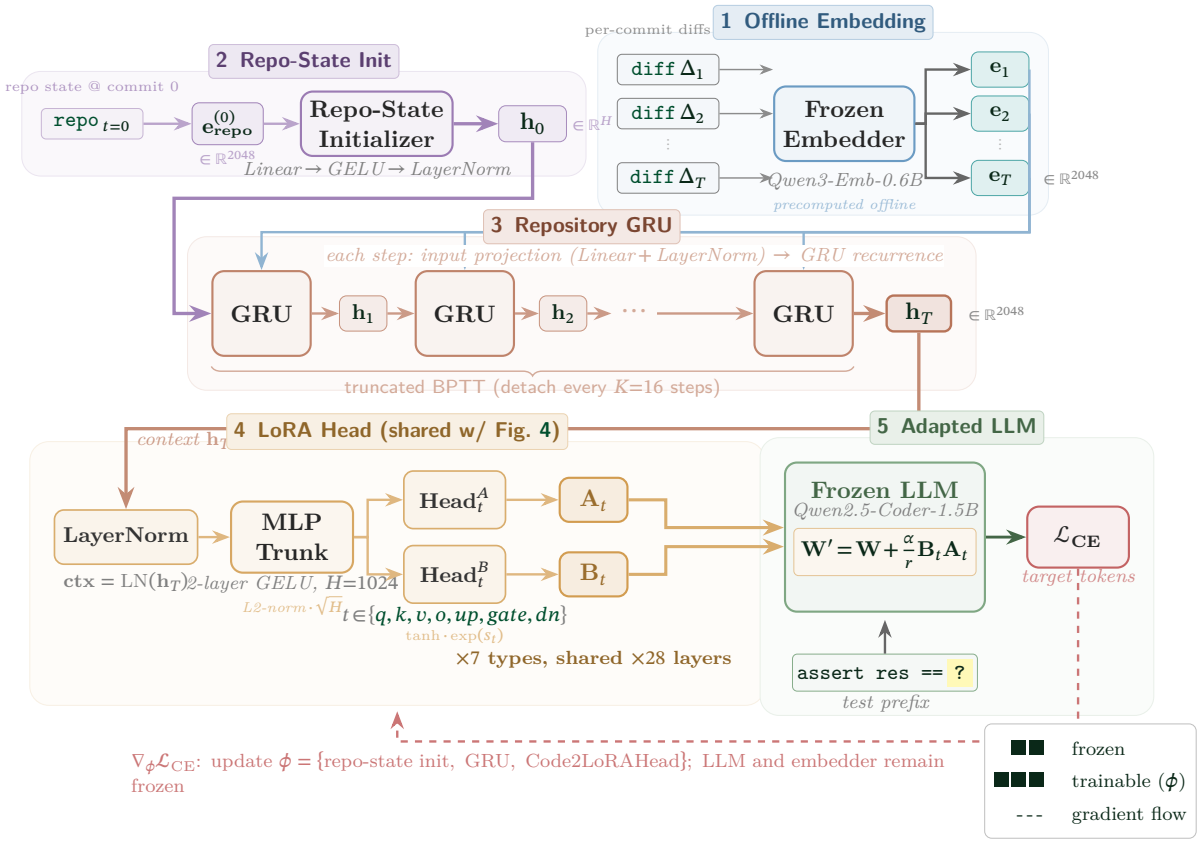

FIGURE 5 Detailed Code2LoRA-Evo architecture and training procedure. **(1)** Per-commit production-code diffs Δ_t and the initial repository snapshot are encoded by the shared frozen embedder into 2048-dim vectors $\{\mathbf{e}_t\}_{t=1}^{T}$ and $\mathbf{e}^{(0)}_{\text{repo}}$; the resulting embeddings are stored in the dataset. **(2)** A small repo-state initializer (Linear → GELU → LayerNorm) maps the static snapshot $\mathbf{e}^{(0)}_{\text{repo}}$ to the initial hidden state $\mathbf{h}_0 \in \mathbb{R}^{2048}$. **(3)** A 1-layer GRU walks the chronological diff sequence; each step projects $\mathbf{e}_t$ with a Linear + LayerNorm and applies the GRU recurrence to produce $\mathbf{h}_t$. Truncated BPTT detaches the hidden state every K=16 steps. **(4)** The final state $\mathbf{h}_T$ is fed (after LayerNorm) into Code2LoRA-Evo's LoRA-generation projection head (analogous in design to Code2LoRA-Static's; Figure 4): a 2-layer GELU trunk with L2-norm rescaling, plus per-module-type Head^A_m/Head^B_m output heads with $\tanh \cdot \exp(s_m)$ scaling. The resulting $(\mathbf{A}_m, \mathbf{B}_m)$ are shared across all 28 transformer layers per type. **(5)** Generated LoRAs are injected into the frozen LLM ($\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{B}_m\mathbf{A}_m$); training minimizes the cross-entropy loss on the assertion target. Gradients (dashed red) flow through the projection head, GRU, and repo-state initializer; the LLM and embedder stay frozen.

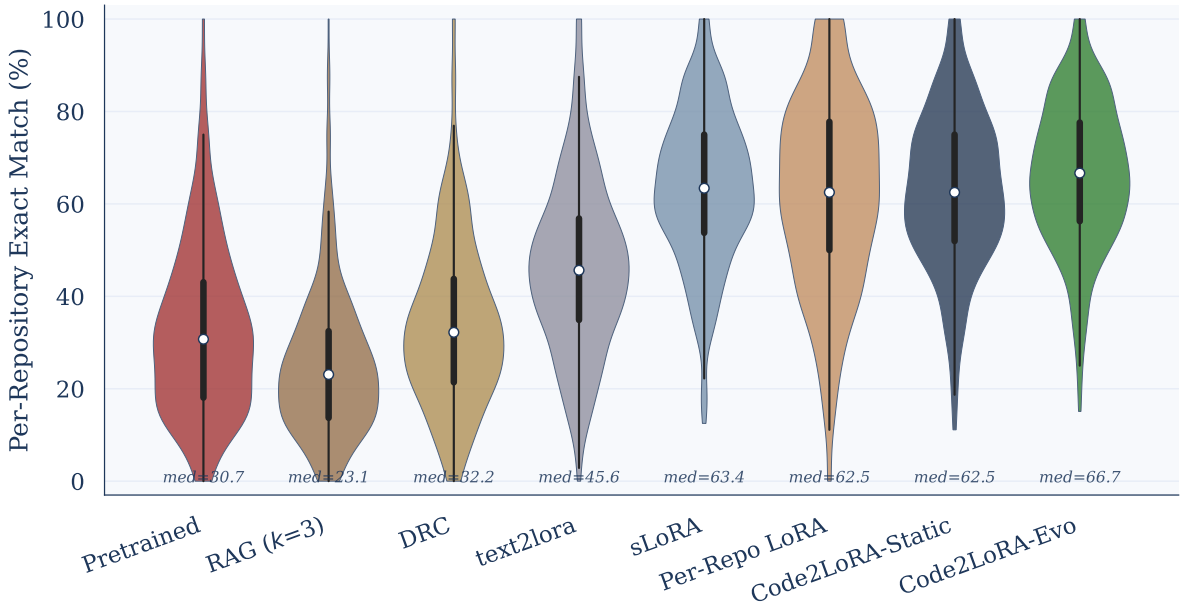

FIGURE 6 Per-repository EM distribution on the IR-test split of RepoPeftBench (Table 3 checkpoints; n=389 repositories common to all methods). Each violin shows the full distribution of per-repository EM for one method; the inner box reports the IQR and the white dot marks the median. Code2LoRA-Static (median 62.5%, σ=16.8) and Code2LoRA-Evo (median 66.7%, σ=15.8) achieve consistently high performance with substantially lower variance than per-repo LoRA (median 62.5%, σ=20.9); per-repo LoRA falls below the pretrained baseline on 10.5% of repositories versus only 1.3% and 1.8% for Code2LoRA-Static and Code2LoRA-Evo, demonstrating the regularizing effect of cross-repository knowledge transfer.

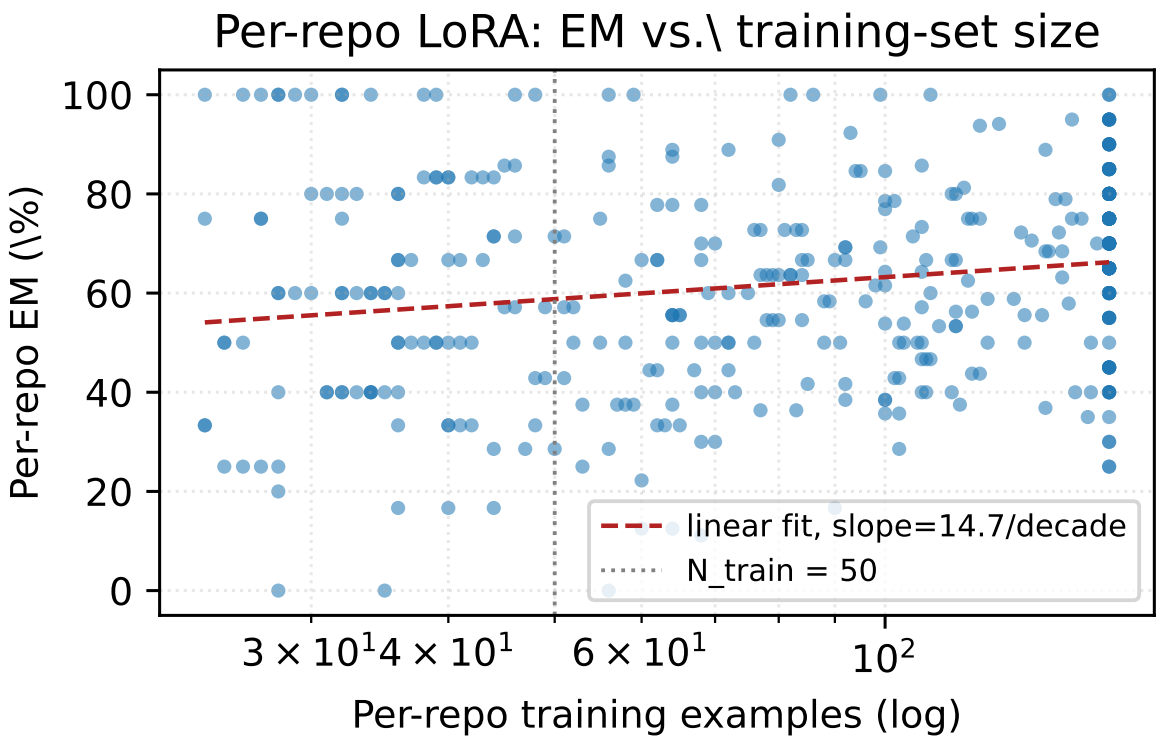

FIGURE 7 Per-repo LoRA EM vs. training-set size on IR test. Repositories with fewer than 50 training pairs frequently underperform the IR-test pretrained baseline (46.8%), while Code2LoRA-Static maintains stable performance regardless of per-repo data availability.

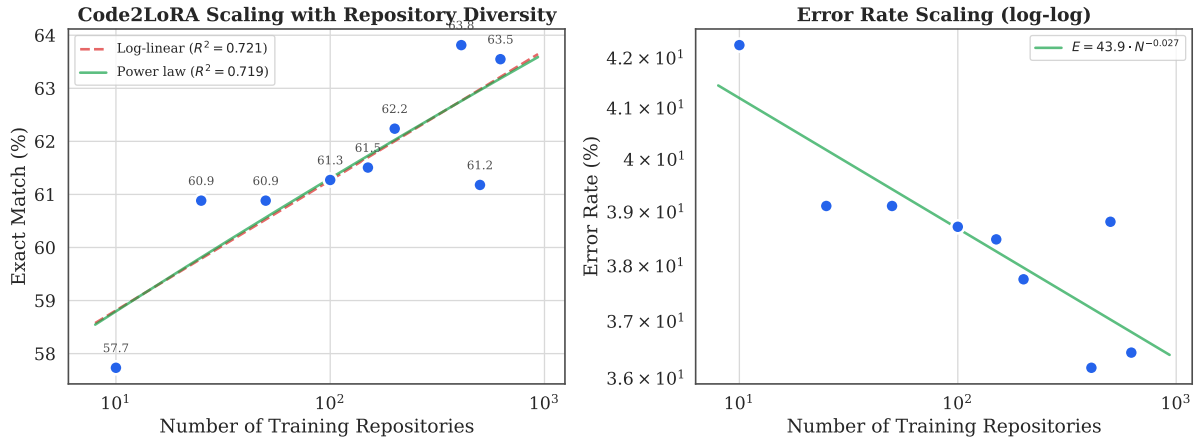

FIGURE 8 CR-test EM as a function of training repository count. Code2LoRA-Static benefits from repository diversity, with performance improving log-linearly.

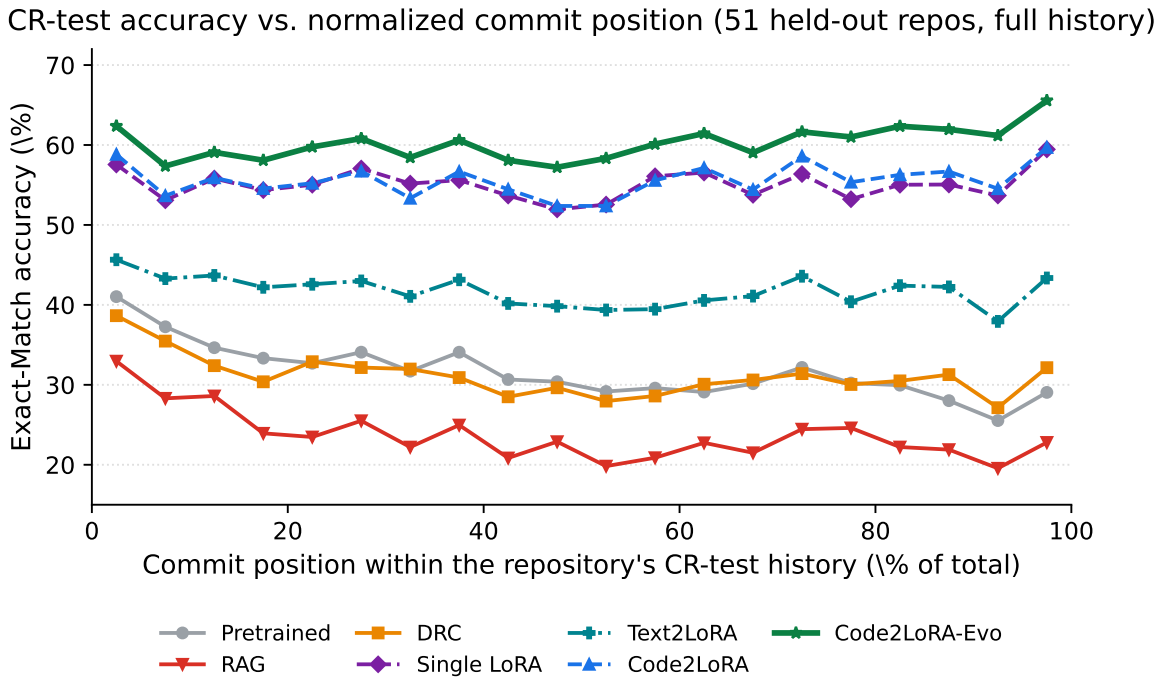

FIGURE 9 CR-test exact-match vs. normalized commit position (51 held-out repositories, commit-derived prefixes). Each repository's timeline is scaled to 0–100%; points are qna-weighted means per 5% bin.

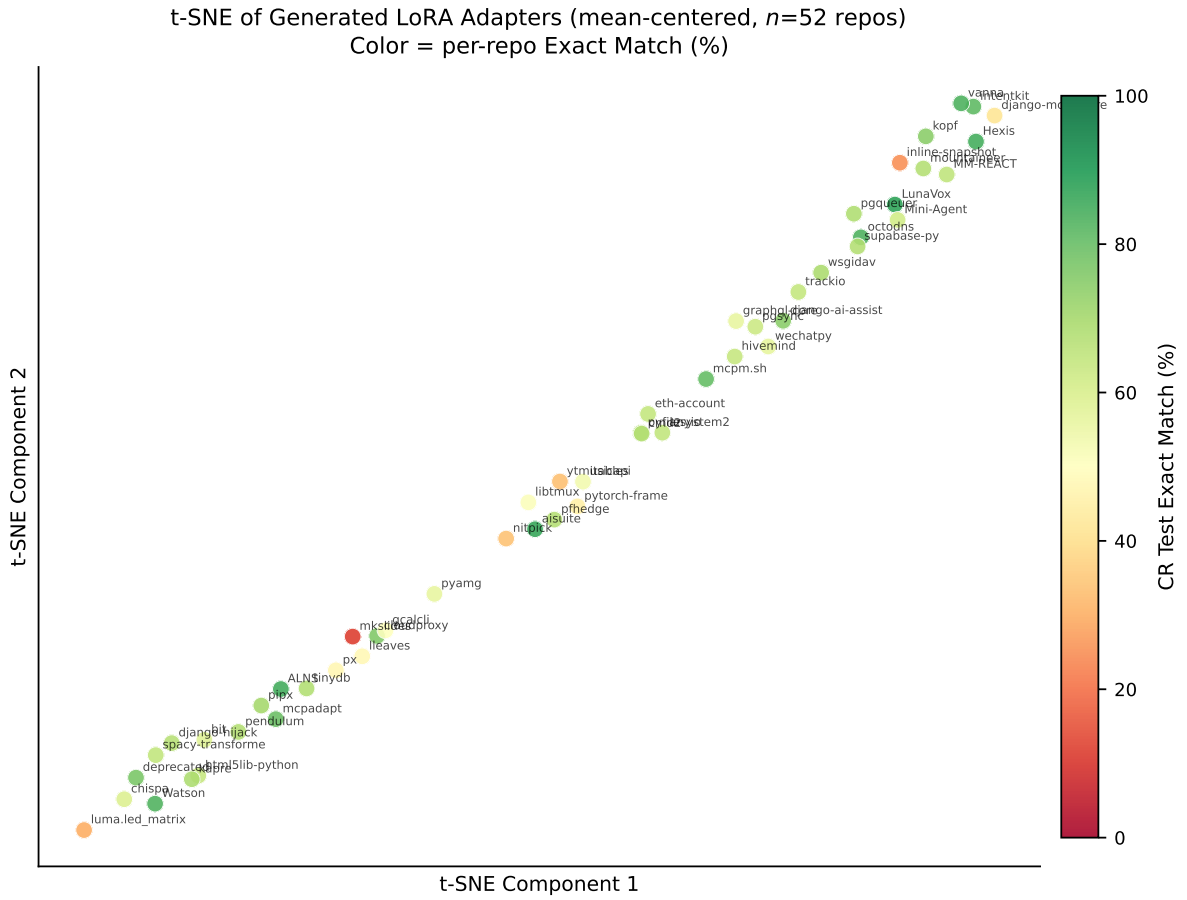

FIGURE 10 t-SNE of generated LoRA adapters for 52 CR-test repositories (PCA pre-reduction to 50 dims, then t-SNE). Color indicates per-repo Exact Match (%). Repositories with similar codebases tend to cluster together, and clusters show coherent EM ranges, demonstrating that the hypernetwork learns a smooth, semantically meaningful adapter manifold.

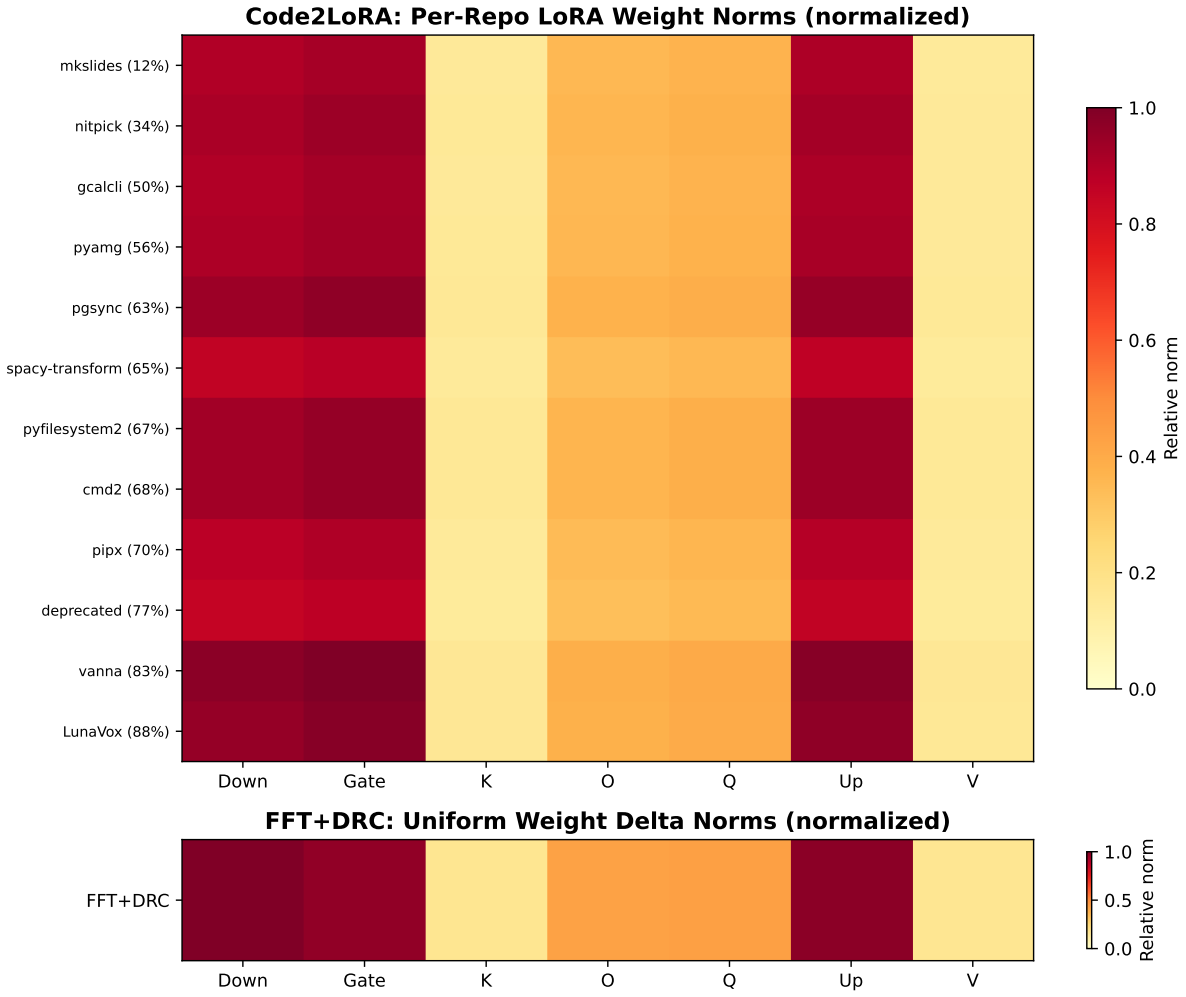

FIGURE 11 Comparison of per-module weight norms. Top: Code2LoRA-Static generates repo-specific LoRA adapters with varying weight distributions across module types. Bottom: FFT+DRC applies a uniform weight delta. Code2LoRA-Static's structured, repo-specific adaptations explain its stronger cross-repo performance.

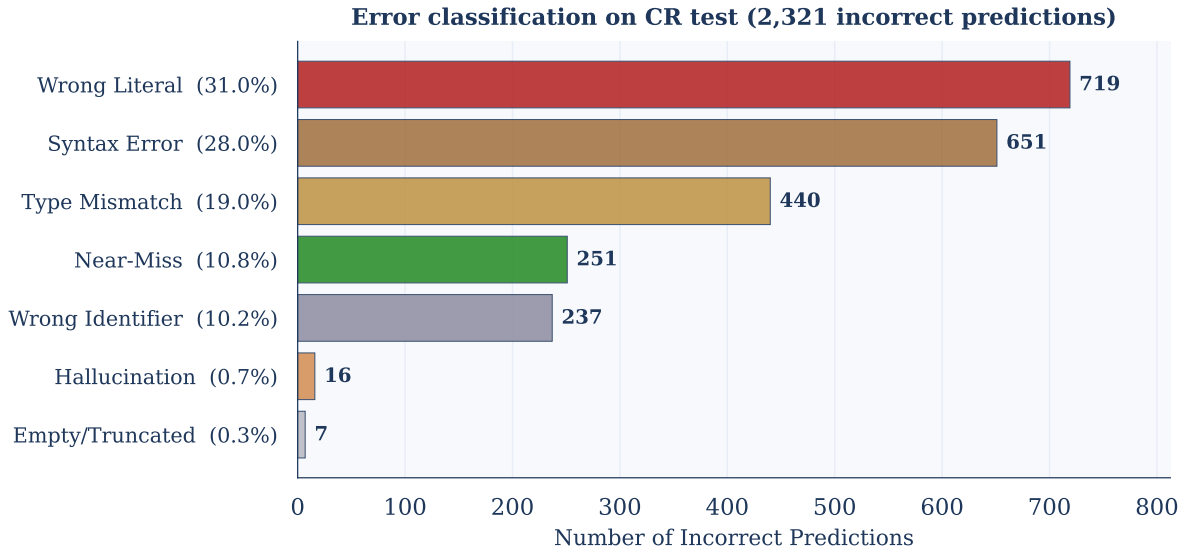

FIGURE 12 Error classification of Code2LoRA-Static failures on CR test (2,321 incorrect predictions), following a LiveCodeBench-inspired taxonomy. Wrong literal (31.0%), syntax error (28.0%), type mismatch (19.0%), near-miss (10.8%), wrong identifier (10.2%); hallucinations and empty outputs are $< 1\%$ each.

(a) *inline-snapshot* (CR) Success

```
s2 = s.run(reported_flag)
assert s2.source ==
    (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Reference	`s2.source`
Code2LoRA-Static	`s2.source`
FFT	`s.source`
Pretrained	`s.source`

Code2LoRA-Static captures the `s2` naming pattern; baselines default to `s`.

(b) *ALNS* (CR) Success

```
select.update(Zero(), 0, 0, 1)
assert_almost_equal(
select.destroy_weights[0],
    (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Reference	`expected[0])`
Code2LoRA-Static	`expected[0])`
FFT	`expected)`
Pretrained	`1)`

The repo uses `expected[i]` arrays for ground truth.

FIGURE 13 Qualitative examples from CR test. Each panel shows a test prefix with the completion target (`???`), ground-truth reference, and model predictions. **(a)**–**(b)**: Code2LoRA-Static correctly infers repo-specific identifiers and conventions that pretrained Qwen2.5-Coder and full fine-tuning miss.

Detailed qualitative example: nolar/kopf (Code2LoRA-exclusive, CR test, commit-derived)

QnA metadata

Repository	nolar/kopf	Commit SHA	d848601b0df0…
Commit position	19.2% (55 / 287)	Python files	131
Repo size	∼423K chars / ∼120K tok	Assertion family	pytest.raises (exception class)
Test location	tests/basic-structs/test_resource.py:7:9		

Test prefix (model input)

```
import pytest
from kopf.structs.resources import Resource

...

def test_no_args():
with pytest.raises( (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Retrieved repository context

DRC (import-resolved, 4K-token budget):

```
# kopf/structs/resources.py
class Resource(NamedTuple):
group: str
version: str
plural: str
@property
def name(self):
return f'{self.plural}.{self.group}'
```

RAG@3 (top-3 retrieved 512-token chunks): surfaces the identical `Resource NamedTuple` definition plus two unrelated chunks (truncated; full text in supplementary).

Per-method predictions and exact-match outcome

Method	Prediction	EM
Reference	`TypeError)`	
Pretrained (Qwen2.5-Coder-1.5B)	`ValueError)`	
RAG ($k=3$)	`ValueError)`	
Dependency-Resolved Context	`ValueError)`	
Single LoRA (sLoRA)	`ValueError)`	
Code2LoRA-Static	`TypeError)`	
Code2LoRA-Evo	`TypeError)`	

The retrieved context surfaces the exact `Resource NamedTuple` with three required fields, so the evidence to deduce that `Resource()` with zero arguments raises `TypeError` is in the prompt. Yet pretrained, RAG, DRC, and sLoRA all default to `ValueError`—the more common `pytest.raises` idiom—showing that input-side methods do not reliably execute the type-level reasoning hop even when the relevant evidence has been retrieved. Both Code2LoRA variants predict `TypeError` because the repository's `NamedTuple`-vs-class conventions were distilled into the LoRA-generation step.

Detailed qualitative example: fla-org/flash-linear-attention (IR test, commit-derived)

QnA metadata

Repository	fla-org/flash-linear-attention	Commit SHA	d62e316ea88b…
Commit position	277 / 409 (training-window)	In-repo split	train
Assertion family	assert_close(...) – repository utility comparing two tensors with a tolerance ratio		
Test location	tests/ops/test_kda.py::test_naive_chunk, line 73		

Test prefix (model input, trimmed)

```
from fla.ops.kda.naive import naive_chunk_kda, naive_recurrent_kda
from fla.utils import IS_INTEL_ALCHEMIST, assert_close, device
...
def test_naive_chunk(B, T, H, D, scale, gate_logit_normalizer, dtype):
...
ref, ref_ht = naive_recurrent_kda(q=..., k=..., v=v.clone(),
g=g.clone(), beta=beta.clone(),
scale=scale, initial_state=h0.clone(),
output_final_state=True)
tri, tri_ht = naive_chunk_kda(q=..., k=..., v=v.clone(),
g=g.clone(), beta=beta.clone(),
scale=scale, initial_state=h0.clone(),
output_final_state=True)
assert_close("o",  ref,    tri,    0.005)
assert_close("ht", ref_ht, tri_ht, (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Retrieved repository context (trimmed)

DRC (import-resolved):

```
# fla/ops/kda/naive.py
def naive_recurrent_kda(q, k, v, g, beta,
scale=None, initial_state=None,
output_final_state=False): ...
def naive_chunk_kda(q, k, v, g, beta,
scale=None, initial_state=None,
output_final_state=False, chunk_size=64): ...

# fla/utils.py -- this is the discriminating signature
def assert_close(prefix, ref, tri, ratio,
warning=False, err_atol=1e-6):
...   # the 4th positional argument is the tolerance ``ratio''
```

RAG@3 (top-3 retrieved chunks): unrelated benchmarks/ops/benchmark_kda.py kernel-benchmarking loop (truncated; no assert_close signature is included).

Per-method predictions and exact-match outcome

Method	Prediction	EM
Reference	0.005)	
Code2LoRA-Static	0.005)	
Code2LoRA-Evo	0.005)	
RAG ($k=3$)	0.005)	
Dependency-Resolved Context	0.005)	

Detailed qualitative example: se2p/pynguin (IR test, commit-derived)

QnA metadata

Repository	se2p/pynguin	Commit SHA	3f25634f7ec7…
Commit position	932 / 1144 (late history)	In-repo split	val
Assertion family	bare assert, comparing a registry return value to an auto-increment integer id		
Test location	tests/instrumentation/test_tracer.py::test_line_registration, line 61		

Test prefix (model input, trimmed)

```
from pynguin.instrumentation.tracer import (
LineMetaData, SubjectProperties, ...
)
...
def test_line_registration(subject_properties: SubjectProperties):
assert subject_properties.register_line(
LineMetaData(0, "foo", 42)) == 0
assert subject_properties.register_line(
LineMetaData(0, "foo", 43)) == (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Retrieved repository context (trimmed)

DRC (import-resolved):

```
# src/pynguin/instrumentation/tracer.py
class LineMetaData:
"""Stores meta data of a line."""
code_object_id: int
file_name: str
line_number: int
...
```

RAG@3 (top-3 retrieved chunks; the discriminating method body):

```
# src/pynguin/instrumentation/tracer.py
class SubjectProperties:
existing_lines: dict[int, LineMetaData] = field(default_factory=dict)
...
def register_line(self, meta: LineMetaData) -> int:
if meta not in self.existing_lines.values():
line_id = len(self.existing_lines)   # auto-increment
self.existing_lines[line_id] = meta
else:
...   # return the existing id for an already-registered line
return line_id
```

Per-method predictions and exact-match outcome

Method	Prediction	EM
Reference	1	
Code2LoRA-Static	1	
Code2LoRA-Evo	1	
RAG ($k=3$)	1	
Dependency-Resolved Context	1	
Text2LoRA	1	

Detailed qualitative example: beartype/beartype (IR test, commit-derived)

QnA metadata

Repository	beartype/beartype	Commit SHA	5f8778d6ba44…
Commit position	902 / 1014 (late history)	In-repo split	test
Assertion family	assert <expr> is ? – the discriminating slot is a boolean identity literal		
Test file	beartype_test/a00_unit/a50_check/a60_error/a90_main/test_errorget.py, line 197		
Test function	test_get_func_pith_violation_conf_is_color		

Test prefix (model input, trimmed)

```
from beartype import BeartypeConf
from beartype._check.error.errmain import get_func_pith_violation
from beartype._util.text.utiltextansi import is_str_ansi
...
def test_get_func_pith_violation_conf_is_color() -> None:
...
# Violation configured to contain ANSI escape sequences.
violation = get_func_pith_violation(
call_meta=minify_decor_meta_kwargs(
func=she_drew_back, conf=BeartypeConf(is_color=True)),
**kwargs)
# Assert this violation message contains ANSI escape sequences.
assert is_str_ansi(str(violation)) is (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Retrieved repository context (trimmed)

DRC (import-resolved; *includes the discriminating signature*):

```
# beartype/_check/error/errmain.py
def get_func_pith_violation(call_meta, pith_name,
pith_value, **kwargs) -> Exception: ...
# beartype/_check/metadata/call/callmetadecormin.py
def minify_decor_meta_kwargs(...): ...

# beartype/_util/text/utiltextansi.py -- this is the discriminating signature
def is_str_ansi(text: str) -> bool:
"""True only if the passed text contains one or more ANSI escape sequences."""
...
return _ANSI_REGEX.search(text) is not None
```

RAG@3 (top-3 retrieved chunks): `get_func_pith_violation` body + `checkmake.py` helpers (truncated). *Neither chunk includes the `is_str_ansi` signature, so RAG never sees the `-> bool` return type.*

Per-method predictions and exact-match outcome

Method	Prediction	EM
Reference	True	
RAG ($k=3$)	1	
Dependency-Resolved Context	True	
Code2LoRA-Static	True	
Code2LoRA-Evo	True	
Text2LoRA	True	

Detailed qualitative example: agronholm/apscheduler (IR test, commit-derived)

QnA metadata

Repository	agronholm/apscheduler	Commit SHA	e4b1db1dcb8d…
Commit position	353 / 1207 (mid-history)	In-repo split	val
Assertion family	assert <expr> is <enum-member> – the slot is a JobOutcome enum value		
Test location	tests/test_datastores.py::test_reap_abandoned_jobs, line 857		

Test prefix (model input, trimmed)

```
from apscheduler import Job, JobOutcome, Task, ...
from apscheduler.datastores.base import BaseExternalDataStore
...
# Earlier in the same test module (line 809) -- same target pattern:
#     assert result.outcome is JobOutcome.abandoned

async def test_reap_abandoned_jobs(datastore: DataStore, ...) -> None:
task = Task(id="task1", func="...", job_executor="async")
await datastore.add_task(task)
job = Job(task_id="task1", executor="async",
result_expiration_time=timedelta(seconds=30))
await datastore.add_job(job)
await datastore.reap_abandoned_jobs("testscheduler")
jobs = await datastore.acquire_jobs("testscheduler", ..., 1)
assert len(jobs) == 1
await datastore.reap_abandoned_jobs("testscheduler")
assert not await datastore.get_jobs()
abandoned_job_result = await datastore.get_job_result(jobs[0].id)
assert abandoned_job_result.outcome is (*@\colorbox{yellow!40}{\textbf{???}}@*)
```

Retrieved repository context (trimmed)

DRC (import-resolved; *the answer is literally in a docstring*):

```
# src/apscheduler/abc/_datastore.py
class DataStore(metaclass=ABCMeta):
@abstractmethod
async def reap_abandoned_jobs(self, scheduler_id: str) -> None:
"""Find jobs marked as acquired by the given scheduler ID and
release them with the outcome of :attr:`~JobOutcome.abandoned`."""
...
# src/apscheduler/_structures.py
class Job:
id: UUID; task_id: str; ...
```

RAG@3 (top-3 retrieved chunks; *the enum class is exposed but `.abandoned` itself is not*):

```
# src/apscheduler/abc/_datastore.py (truncated overload)
class DataStore(metaclass=ABCMeta):
async def reap_abandoned_jobs(self, scheduler_id: str) -> None: ...

# src/apscheduler/_events.py
class JobReleased(SchedulerEvent):
""":param outcome: the outcome of the job
:param exception: if ``outcome`` is :attr:`JobOutcome.error`."""   # only the .error member
outcome: JobOutcome = attrs.field(converter=as_enum(JobOutcome))
...
```

Per-method predictions and exact-match outcomes